

缶サットプログラム講習会 2026

(株)ビジュアル・テクノロジー研究所
山本 三七男

自己紹介

山本 三七男（64才）和歌山県 有田市

略 歴

- ・兵庫県立大学(旧 姫路工業大学)卒、
- ・セイコーインスツル株式会社に入社、
研究開発部門で働く、体温発電の研究
- ・2006年まで、東京大学生産技術研究所
で、マイクロ流体の研究



- **2006年、和歌山にUターン**
ルアリダワークス社設立
ソフト・ハードウェアの委託開発



- **2014年～現在、**
(株)ビジュアル・テクノロジー研究所
東京科学大学承認ベンチャー会社、
人間の視覚の研究をしている。
ソフトウェア開発を担当している。



その他、

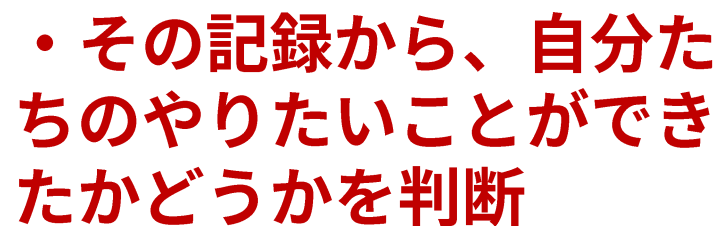
- **きのくにICT 講師、**
- **和歌山県宇宙教育研究会所属**

宇宙甲子園 缶サット部門とは

<http://space-koshien.com/cansat>

高校生が自作した缶サットを打上げ、
上空での放出・降下・着地の過程を通じて、
技術力・創造力を競う競技会です。

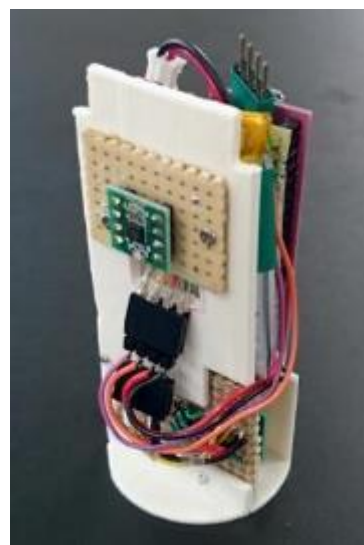
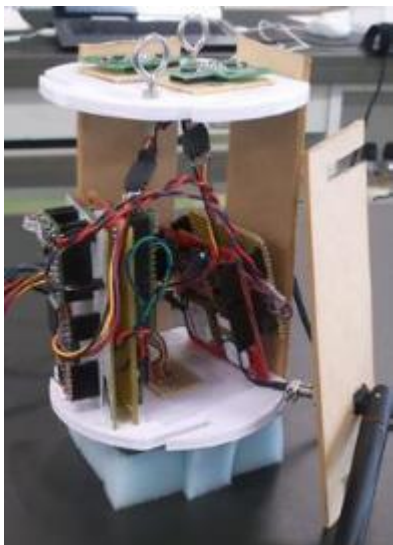
「定められた技能」を競うのではなく、
生徒の皆さんが斬新でオリジナリティーのある
缶サットを作り、
「coolさ」を競います。



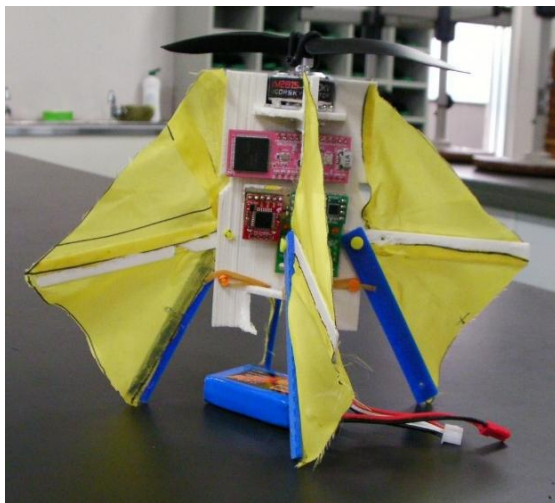
・結果を報告

CanSat: 模擬人工衛星

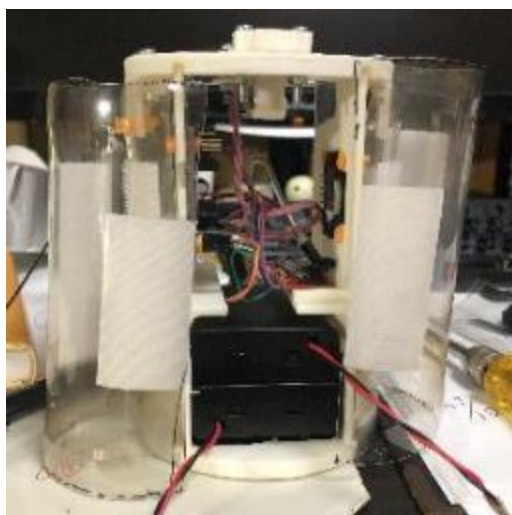
いろいろな缶サット (模擬人工衛星)



いろいろな缶サット (模擬人工衛星)



いろいろな缶サット (模擬人工衛星)



いろいろな缶サット (模擬人工衛星)



<https://youtu.be/XJ6JPZCV5sg?si=329IACluYCSnDd4K&t=44>

部材紹介 はんだ付け

缶サット講習に使用する部材

- マイコン: M5Stamp S3A



<https://logikara.blog/stamps3-init/>

- GNSS: GPS V1.1 (AT6668 + MAX2659)



<https://www.switch-science.com/products/10037>

缶サット講習に使用する部材

- 各種センサ: ENV.III (SHT30+QMP6988)



SHT30: 温度センサ、湿度センサ
QMP6988: 絶対気圧センサ

<https://www.switch-science.com/products/7254>

- 各種センサ: IMU (MPU6886)



MPU6886: 3軸重力加速度、角速度センサ、温度

<https://www.switch-science.com/products/6623>

缶サット講習に使用する部材

- 光センサ: UNIT LIGHT [U021]



<https://www.switch-science.com/products/4051>

- 電源: 電池ボックスとスナップ



単3電池 3本を直列に接続

<https://akizukidenshi.com/catalog/g/g102677/>

<https://akizukidenshi.com/catalog/g/g100452/>

缶サット講習に使用する部材

- 小型カメラ: M5Stack用Cam93-5MP Wi-Fiカメラ



<https://www.switch-science.com/products/9923>

- 小型カメラ: T9小型カメラ Wi-Fiカメラ



https://www.amazon.co.jp/dp/B0H28912RS?ref=ppx_yo2ov_dt_b_fed_asin_title

缶サット講習に使用する部材

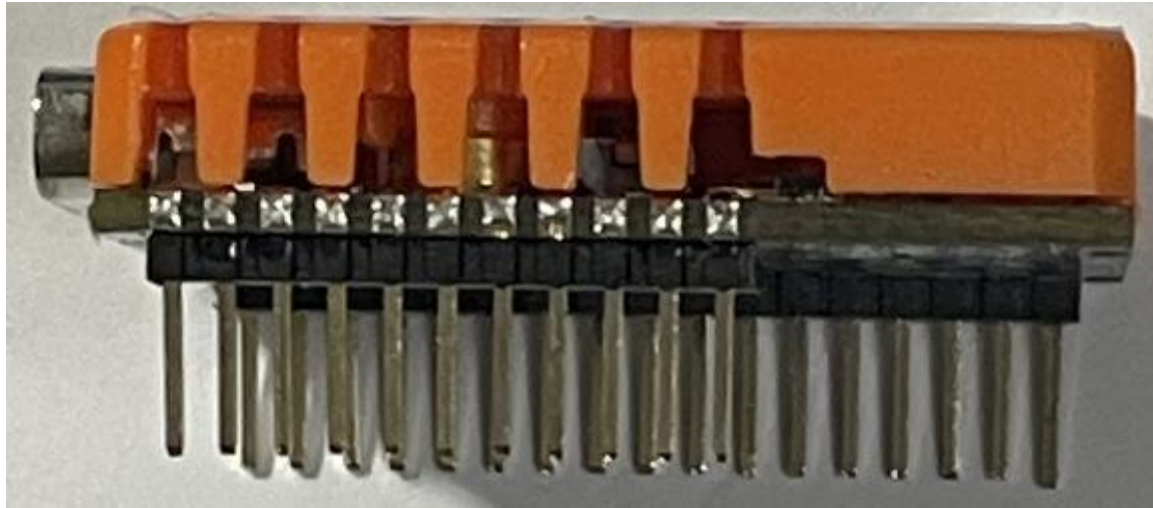
- **CanSat基板: CanSat 2026 TEST Board**



今年、この基板を使います。

部材のはんだ付け

- ・ マイコン: M5Stamp S3A



1.27ピッチで28ピンのはんだ付け済みです。

部材のはんだ付け

• CanSat基板



- Grove端子を向きを間違えないようにはんだ付けします。
- USB Type-Cソケットをはんだ付けします。
- M5 Stamp S3Aのソケットをはんだ付けします。
- 電池スナップを横向き1カ所はんだ付けします。
- 基板の裏にヘッダピンを2カ所はんだ付けします。
- マイクロSDカードスロットを表面はんだします。
- 3.3V出力ピンを1カ所はんだ付けします。

全て、はんだ付け済みです。

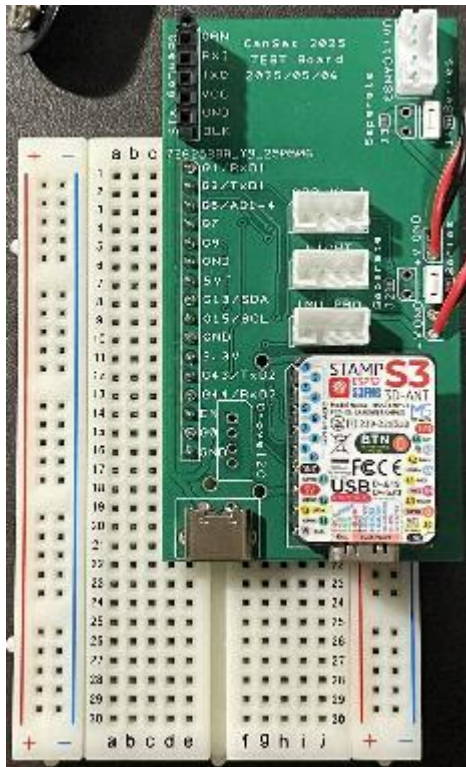
プログラミング

回路の製作

回路基板図面は、CanSat2026図面

・ブレッドボード

今年は、使いません。



電子部品やマイコンをはんだ付け不要で、簡単に接続できる基板です。

<https://www.youtube.com/watch?v=GzaLyulyskA>

まずは、「Lチカ」から

「Lチカ」は、マイコンを使ってLEDを点滅させる基本的なプログラミング練習です。

Lチカ(LED点滅)

USBケーブルを使って、PCとM5StampS3Aを接続してください。

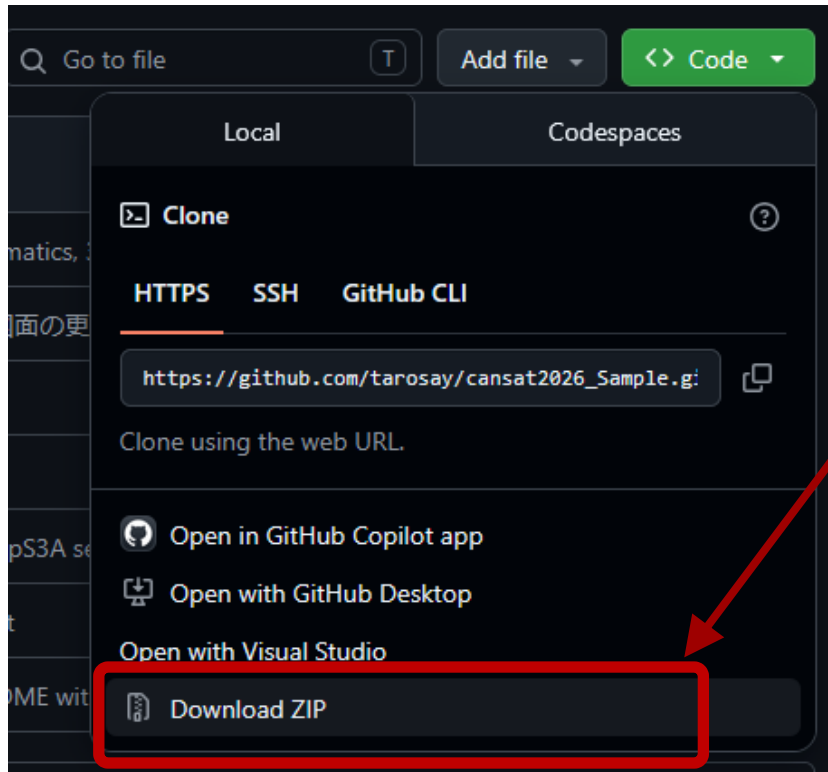
フルカラーLEDが点滅します。

https://github.com/tarosay/cansat2026_Sample/

Githubからサンプルプログラムをダウンロードしてください。



Lチカ(LED点滅)

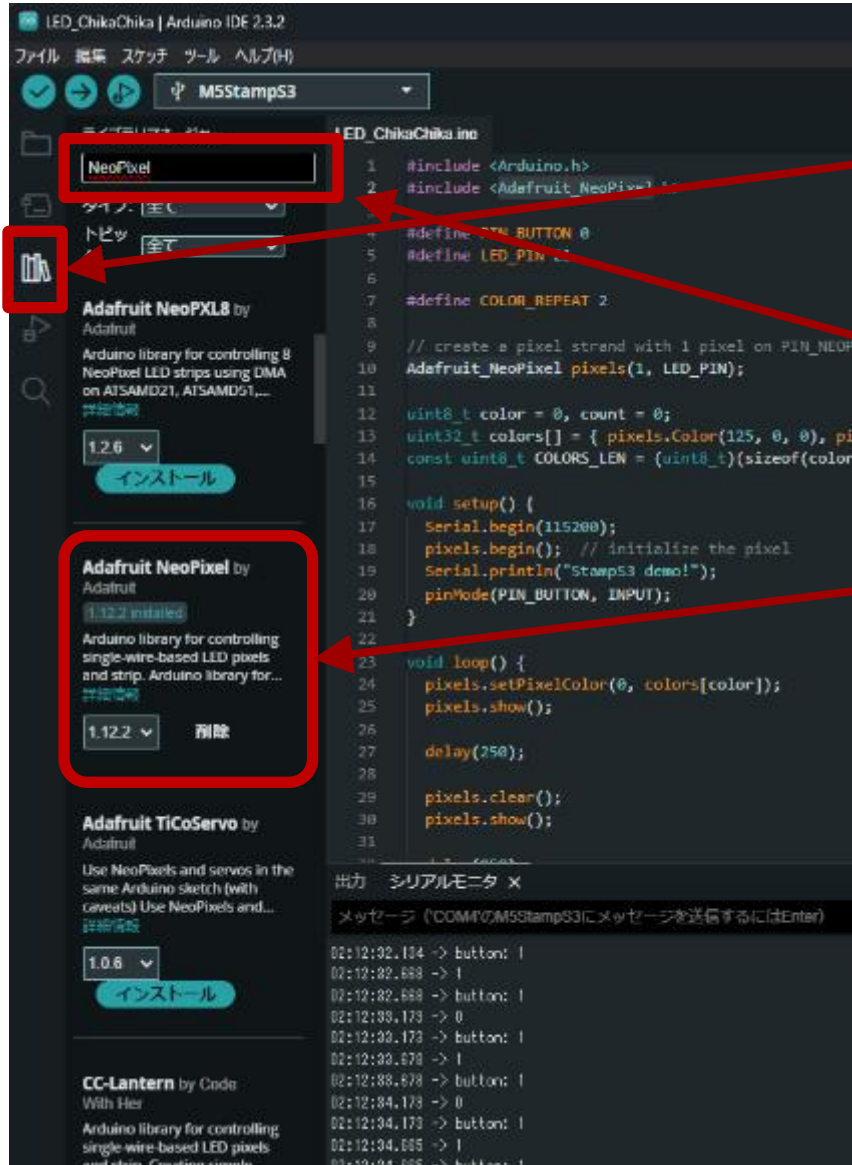


ZIPファイルをダウンロードして解凍する。

M5Stamp_S3A → LED_ChikaChika →
*LED_ChikaChika.ino*をダブルクリック

https://github.com/tarosay/cansat2026_Sample/

Lチカ(LED点滅)



(1) ライブラリアイコンをクリック

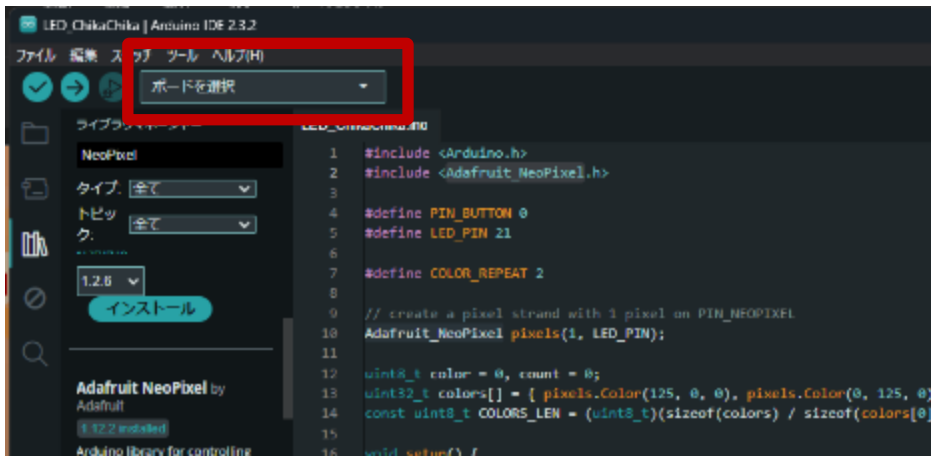
(2) 検索窓に「NeoPixel」といれます。

(3) 出てきた「Adafruit NeoPixel by Adafruit」をインストールします。

これで、Lチカプログラムがビルドできます。

Lチカ(LED点滅)

ボードとCOMポートを選択します。

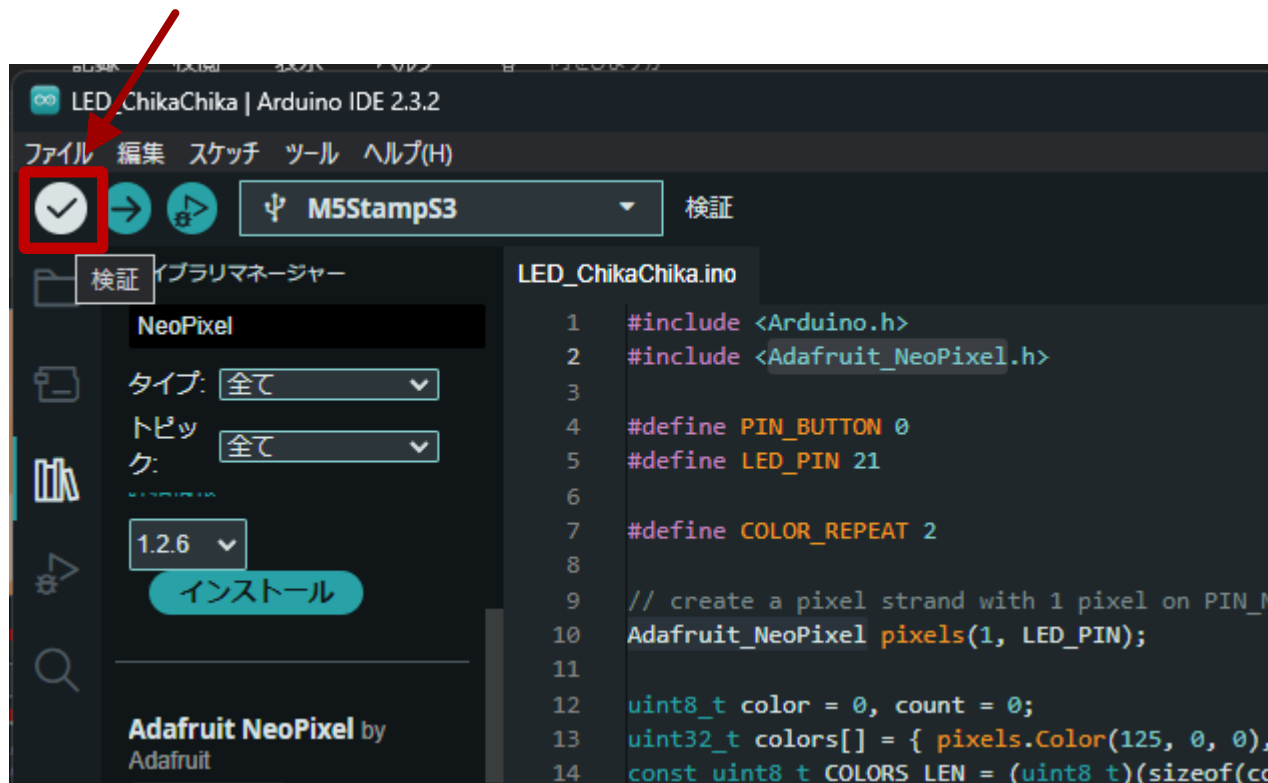


ボード: M5StampS3 M5Stack
を選びます。

ポートは。USBポートを選びます。

Lチ力(LED点滅)

エラーなくビルドできるか、確認できます。



出力に、この表示が出れば、成功です。

次のフォルダのライブラリ **Adafruit NeoPixel** バージョン **1.12.2** を使用中: **D:\Users\minao\Documents\Arduino\libraries\Adafruit_NeoPixel**
"C:\Users\minao\AppData\Local\Arduino15\packages\m5stack\tools\xtensa-esp32s3-elf-gcc\esp-2021r2-patch5-8.4.0/bin/xtensa-esp32s3-elf
 最大 **1310720** バイトのフラッシュメモリのうち、スケッチが **257809** バイト (**19%**) を使っています。
 最大 **327680** バイトの RAM のうち、グローバル変数が **18456** バイト (**5%**) を使っていて、ローカル変数で **309224** バイト使うことができます。

Lチカ(LED点滅)

`Adafruit_NeoPixel pixels(1, LED_PIN);`
が、フルカラーLED(ネオピクセル)の設定で、
LEDの数=1つ、
ピン番号が21番
を設定しています。

`pixels.begin();`
で、ネオピクセル利用開始となります。

`pixels.setPixelColor(0, pixels.Color(125, 0, 0));`
で、0番のLEDに、Red, Green, Blueの割り合いを設定し、色を決めます。
LEDは、0から番号が振られます。今回は1つなので、0番のみです。

`pixels.show();`
で、色が変わります。

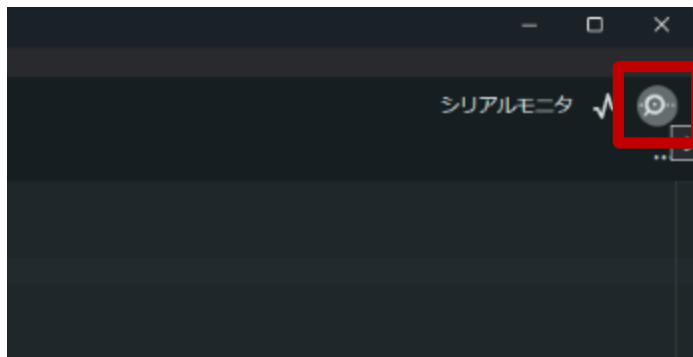
Lチカは、マイコンの状態を示すインジケータとして利用できます。
是非、使い方を覚えてください。

UART(シリアル通信)

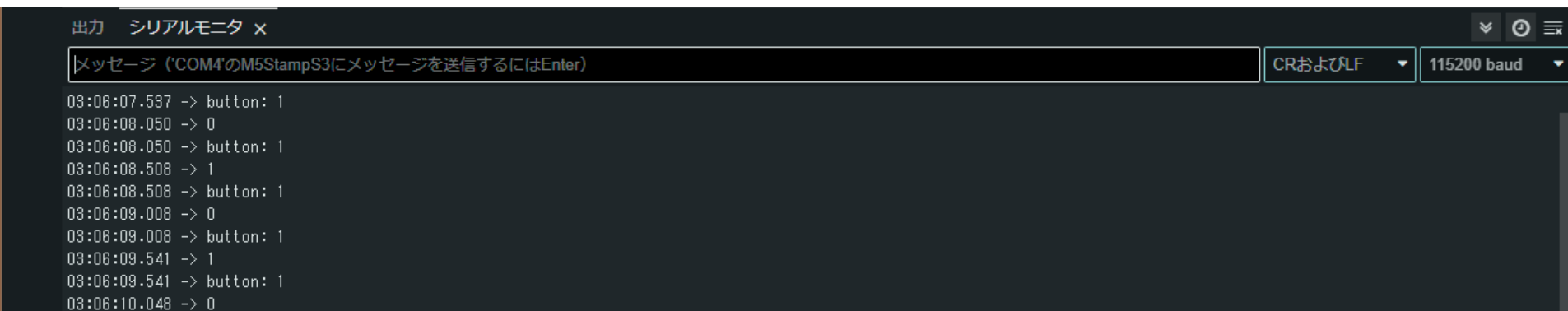
https://www.youtube.com/watch?v=lmkAn_Rr4fM

マイコンとPCは、USBポートをUARTシリアル通信を行い、データをやり取りできます。

左上のシリアルモニタを押します。

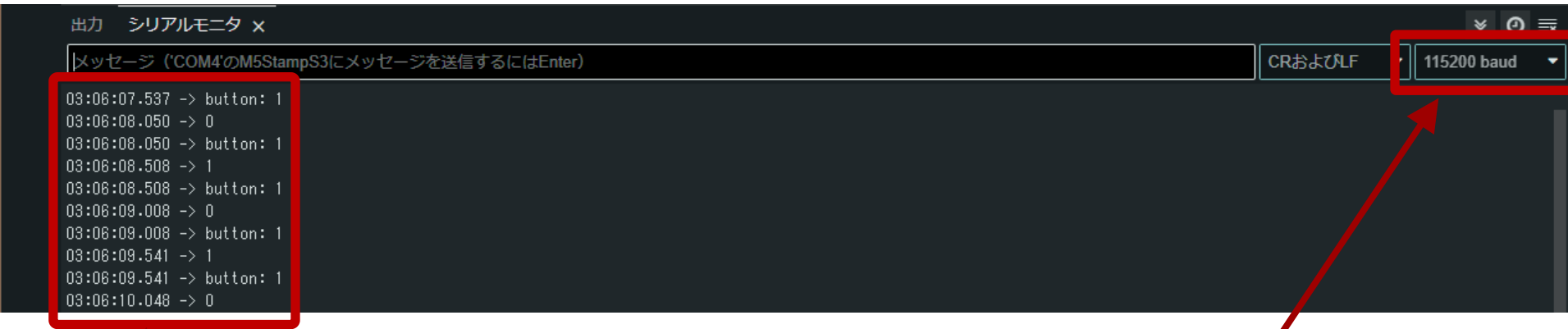


下にシリアルモニタが出てきます。



UART(シリアル通信)

https://youtu.be/lmkAn_Rr4fM?si=nRjkgprTvqCvMePL&t=990



マイコンが送信してきたデータです。

ボーレートを合わせます。

マイコンプログラムのボーレートが、115200ボーになっています。

Serial. が、USBシリアルに設定されています。

```
15
16 void setup() {
17   Serial.begin(115200);
18   pixels.begin(); // initialize the pixel
19   Serial.println("StampS3 demo!");
20   pinMode(PIN_BUTTON, INPUT);
21 }
22
```

UART(シリアル通信)

M5Stamp S3Aには、3つのUARTシリアルを持っています。

Serial.

Serail1.

Serial2.

です。

Serial. USBシリアル

Serial1. G1(受信)/G2(送信)シリアル

Serial2. G44(受信)/G43(送信)シリアル

Serial. は、PCと接続

Serial1. は、OpenLogと接続(今年は使いません)

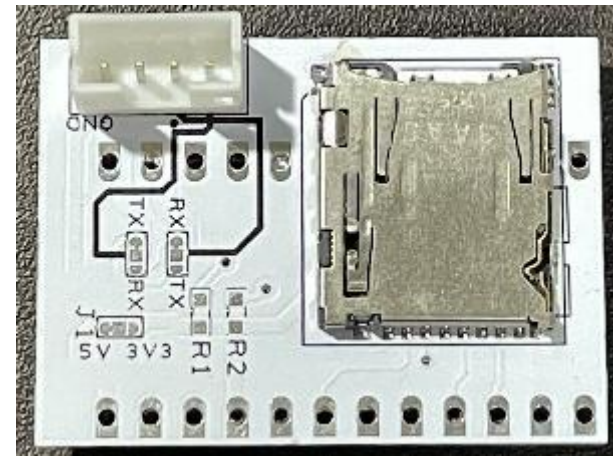
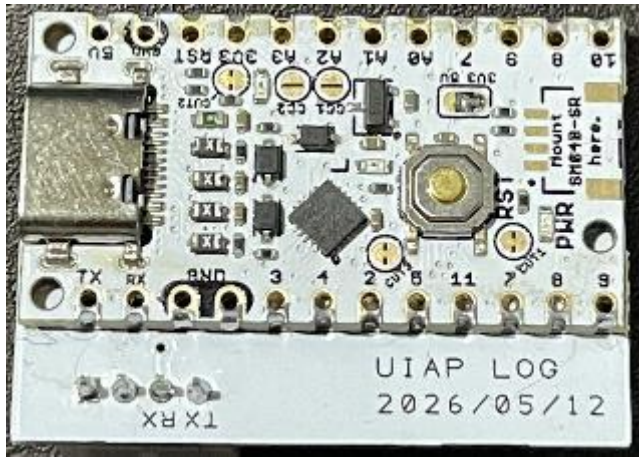
Serial2. は、GPSと接続

となります。

今年は、OpenLogを使わないので、Serial1は、空いています。

UIAP Log(シリアル通信ロガー)

Open Logの代わりに、290円マイコンで製作したOpenLog互換基板を配布します。



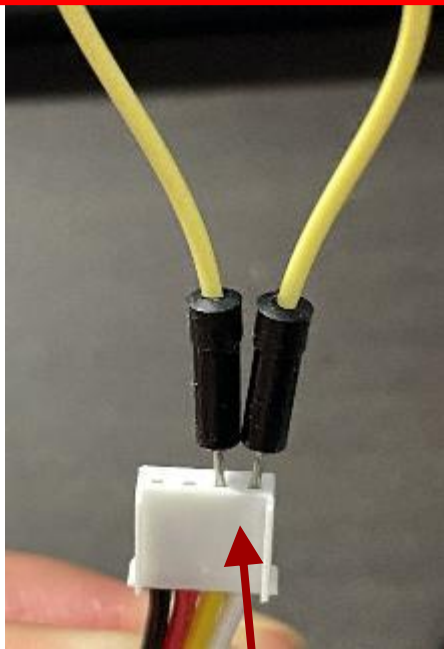
Serial1 のグローブ端子に直結すると、OpenLogと同様の記録をします。

UART(シリアル通信)

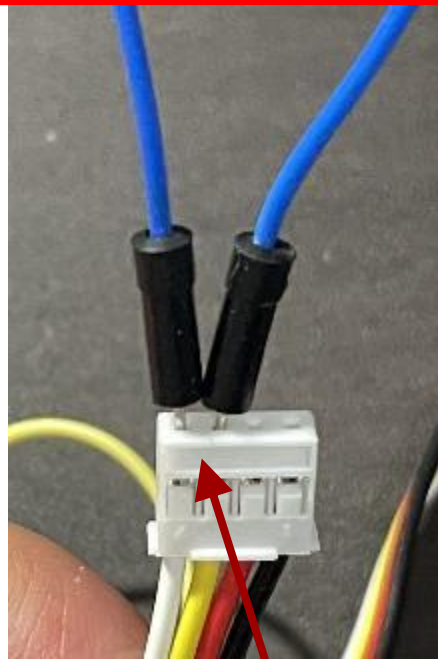
UART通信のループバックテスト

ループバックテストは、送信ピン (TX) を受信ピン (RX) に接続して、送信データが正しく受信されるかを確認するテストです。

赤線と黒線は電源なので、絶対にショートさせないように！



43番と44番を接続



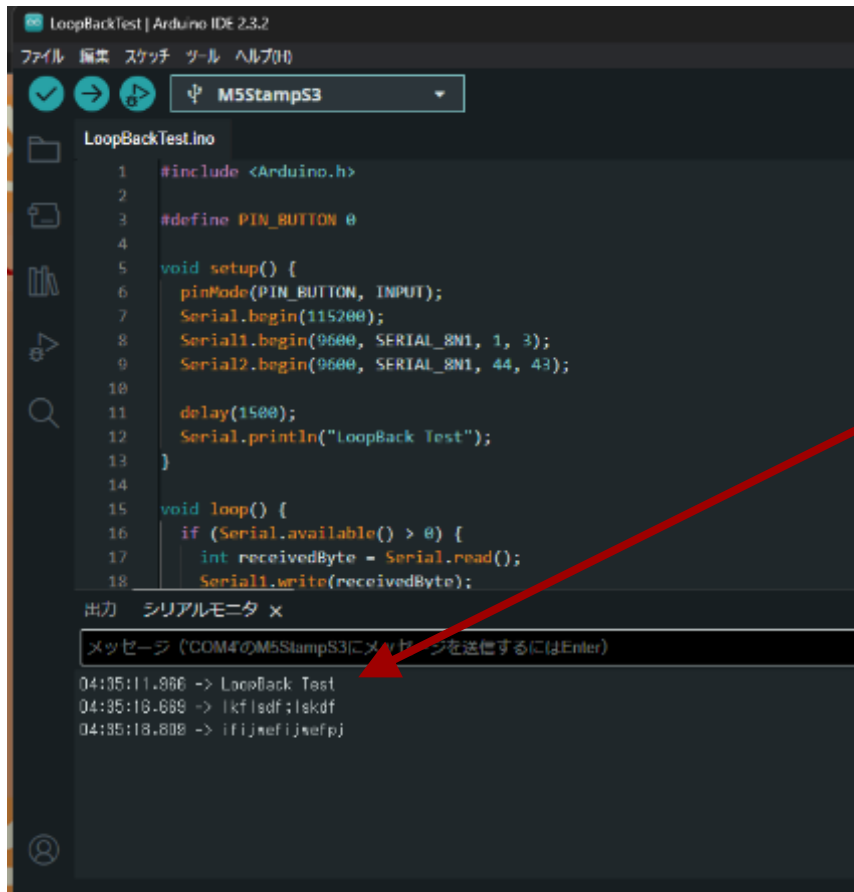
1番と3番を接続



UART(シリアル通信)

UART通信のループバックテスト

LoopBackTest



The screenshot shows the Arduino IDE interface. The top menu bar includes 'ファイル', '編集', 'スケッチ', 'ツール', and 'ヘルプ(10)'. The toolbar has icons for checking, running, and uploading, along with a dropdown menu showing 'M5StampS3'. The code editor displays the 'LoopBackTest.ino' file with the following code:

```
1 #include <Arduino.h>
2
3 #define PIN_BUTTON 0
4
5 void setup() {
6   pinMode(PIN_BUTTON, INPUT);
7   Serial.begin(115200);
8   Serial1.begin(9600, SERIAL_8N1, 1, 3);
9   Serial2.begin(9600, SERIAL_8N1, 44, 43);
10
11   delay(1500);
12   Serial.println("LoopBack Test");
13 }
14
15 void loop() {
16   if (Serial.available() > 0) {
17     int receivedByte = Serial.read();
18     Serial1.write(receivedByte);
```

Below the code editor is the '出力 シリアルモニタ' (Output Serial Monitor) window. It shows the following output:

```
04:35:11.800 -> LoopBack Test
04:35:18.689 -> lkflsdf;lskdf
04:35:18.800 -> ifijnef ijnefpj
```

A red arrow points from the text on the right to the input field of the Serial Monitor, which contains the text: 'メッセージ (COM4のM5StampS3にメッセージを送信するにはEnter)'.

シリアルモニタで、入力した文字が返って来るかどうかを確認する。
Serial2. もテストしてください。

UART(シリアル通信)

UART通信のループバックテスト

LoopBackTest.ino

```
4
5 void setup() {
6   pinMode(PIN_BUTTON, INPUT);
7   Serial.begin(115200);
8   Serial1.begin(9600, SERIAL_8N1, 1, 3);
9   Serial2.begin(19200, SERIAL_8N1, 44, 43);
10
11   delay(1500);
12   Serial.println("LoopBack Test");
13 }
14
15 void loop() {
16   if (Serial.available() > 0) {
17     int receivedByte = Serial.read();
18     Serial1.write(receivedByte);
19   }
20
21   if (Serial1.available() > 0) {
22     int receivedByte = Serial1.read();
23     Serial2.write(receivedByte);
24   }
25
26   if (Serial2.available() > 0) {
27     int receivedByte = Serial2.read();
28     Serial.write(receivedByte);
29   }
30   delay(10);
31 }
```

LoopBackTest

テストプログラム説明

キー入力



USBシリアル RxD



シリアル1 TxD



シリアル1 RxD



シリアル2 TxD



シリアル2 RxD



USBシリアル TxD



PCの画面

ループバック

ループバック

UART(シリアル通信)

GNSS(GPS)のUART通信テスト



Serial2_RXD44_TXD43_Sample

Grove端子GPS V1.1に
GPSを接続します。

Serial2 43番,44番に接続
されます。

GPSデータが受信できます

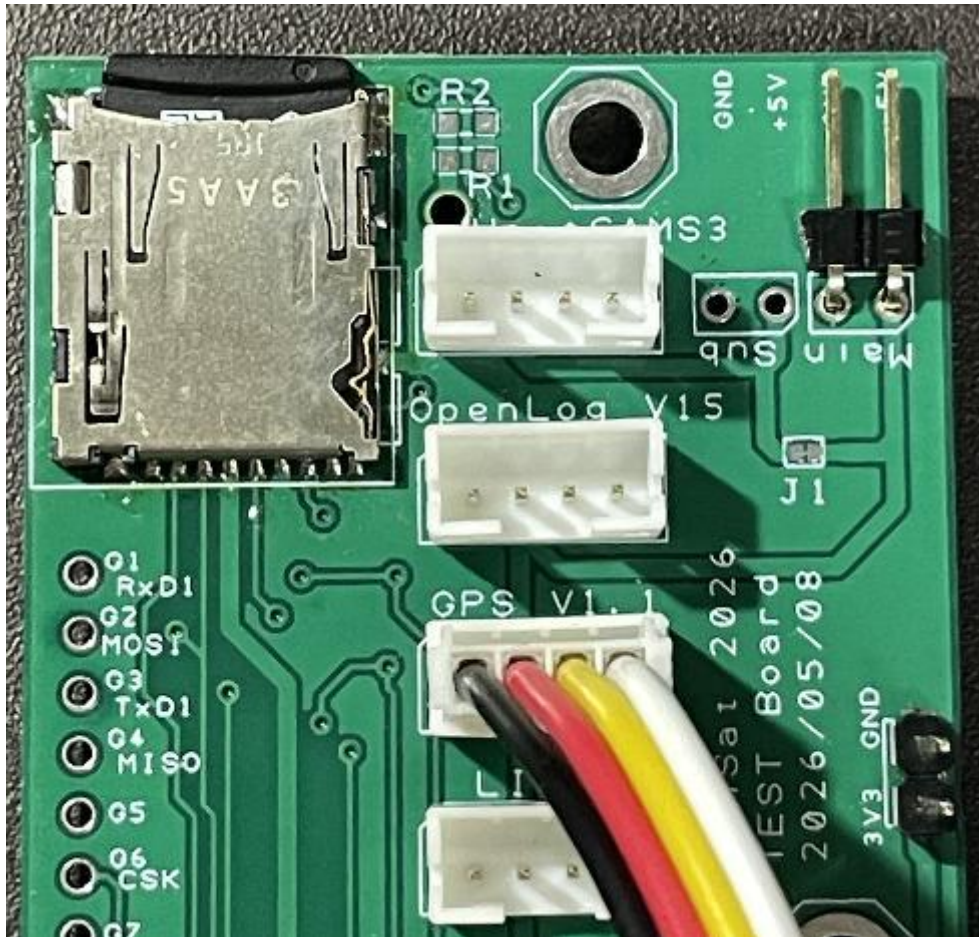
出力 シリアルモニタ ×

メッセージ ('COM5'のM5StampS3にメッセージを送信するにはEnter)

```
007.07912,E,201633.00,H,M*7D
$GNZDA,201633.00,03,06,2025,00,00*7D
$GPTXT,01,01,01,ANTENNA OPEN*25
507.07912,E,201634.00,A,A*79
$GNZDA,201634.00,03,06,2025,00,00*7A
$GPTXT,01,01,01,ANTENNA OPEN*25
507.07912,E,201635.00,A,A*78
$GNZDA,201635.00,03,06,2025,00,00*7B
$GPTXT,01,01,01,ANTENNA OPEN*25
507.07912,E,201636.00,A,A*7B
$GNZDA,201636.00,03,06,2025,00,00*79
```

SDカード保存(SPI通信)

CSVファイル保存テスト

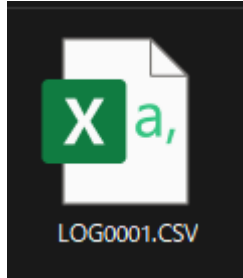


CsvLogStoreは、
`csvLog.write(buffer, len);`
とするだけで、SDカードにデータを
記録します。

SDカードを刺して、
CsvLogStoreTestを書き
込んで、
GPSデータをLog保存して
みましょう。

SDカード保存(SPI通信)

CSVファイル保存テスト



SDカードには、LOGxxxx.CSVファイルができます。
ここにデータが書き込まれるので、Excelで見ることができます。

SDカードへの書き込みは、CsvLogStoreクラスにすべての
プログラムがあります。

CsvLogStoreクラスのヘッダを宣言します。

CsvLogStoreを csvLog という名前で宣言します。

SDに一回で書き込む最大サイズ

CsvLogStoreTest

```
CsvLogStoreTest.ino  CsvLogStore.h  BoardConfig.h  StatusLed.h
1  #include <Arduino.h>
2  #include "BoardConfig.h"
3  #include "StatusLed.h"
4  #include "CsvLogStore.h"
5
6  CsvLogStore csvLog;
7  CsvLogResult csvResult;
8
9  static constexpr size_t GPS_READ_BUFFER_SIZE = 256;
10
```

SDカード保存(SPI通信)

CsvLogStroe.h

```
CsvLogStoreTest.ino  CsvLogStore.h  BoardConfig.h  StatusLed.h
14  class CsvLogStore {
15  public:
16      static constexpr uint16_t LOG_START_NO = 1;
17      static constexpr uint16_t LOG_MAX_NO = 9999;
18
19      CsvLogStore()
20      : _opened(false),
21        _begun(false),
22        _flushEveryRows(0),
23        _flushIntervalMs(0),
24        _rowsSinceFlush(0),
25        _lastFlushMs(0) {
26        _currentPath[0] = '\0';
27      }
28
29      bool begin() {
30        _opened = false;
31        _currentPath[0] = '\0';
32
33        SPI.begin(
34            SD_SPI_SCK_PIN,
35            SD_SPI_MISO_PIN,
36            SD_SPI_MOSI_PIN,
37            SD_SPI_CS_PIN);
38
39        if (!SD.begin(SD_SPI_CS_PIN, SPI, SD_SPI_FREQ)) {
40            _begun = false;
```

今回は、クラスの詳細説明は省略します。

CsvLogStroe.hのコメントを読んで理解してみてください。

分からないことがあれば、質問してください。

SDカード保存(SPI通信)

- `bool begin()` SDカードを初期化します。
- `void setFlushEveryRows(uint16_t rows)` 「何行書いたら自動フラッシュするか」を設定します。
- `void setFlushIntervalMs(uint32_t intervalMs)` 「何ミリ秒たったら自動フラッシュするか」を設定します。
- `bool openNext()` 新しいログファイルを開きます。
- `bool close()` ファイルを閉じます。
- `bool flush()` バッファに溜まっているデータを、実際に SDカードへ書き込みます。
- `CsvLogResult print(const char* text)` 文字列を「改行なし」で書き込みます。
- `CsvLogResult println(const char* text)` 文字列を書き込み、最後に改行 (CR+LF) を付けます。
- `CsvLogResult write(const uint8_t* data, size_t len)` 文字列ではなく、バイト列 (バイナリデータ) をそのまま書き込みます。
- `CsvLogResult writeHeader(const char* line)` CSVのヘッダ行 (1行目の項目名) を書き込みます。
- `CsvLogResult writeRow(const char* line)` CSVのデータ行を1行書き込みます。
- `const char* currentPath()` 現在開いているファイルのパス (例: "/LOG0003.CSV") を返します。
- `bool isOpen()` ファイルが開いているかどうかを返します。
- `bool isBegun()` `begin()` (SDカードの初期化) が成功しているかどうかを返します。
- `uint16_t rowsSinceFlush()` 最後にフラッシュしてから書き込んだ行数を返します。
- `uint32_t lastFlushMs()` 最後にフラッシュした時刻 (`millis()` の値) を返します。

「書き込みが成功したか」と「そのとき自動フラッシュされたか」の2つの情報を返します。

```
struct CsvLogResult {
```

```
bool ok;           // true = 書き込み成功
```

```
bool flushed;    // true = この書き込みのタイミングで自動フラッシュが実行された
```

```
};
```

SDカード保存(SPI通信)

ここで、電池で動かしてみましょう

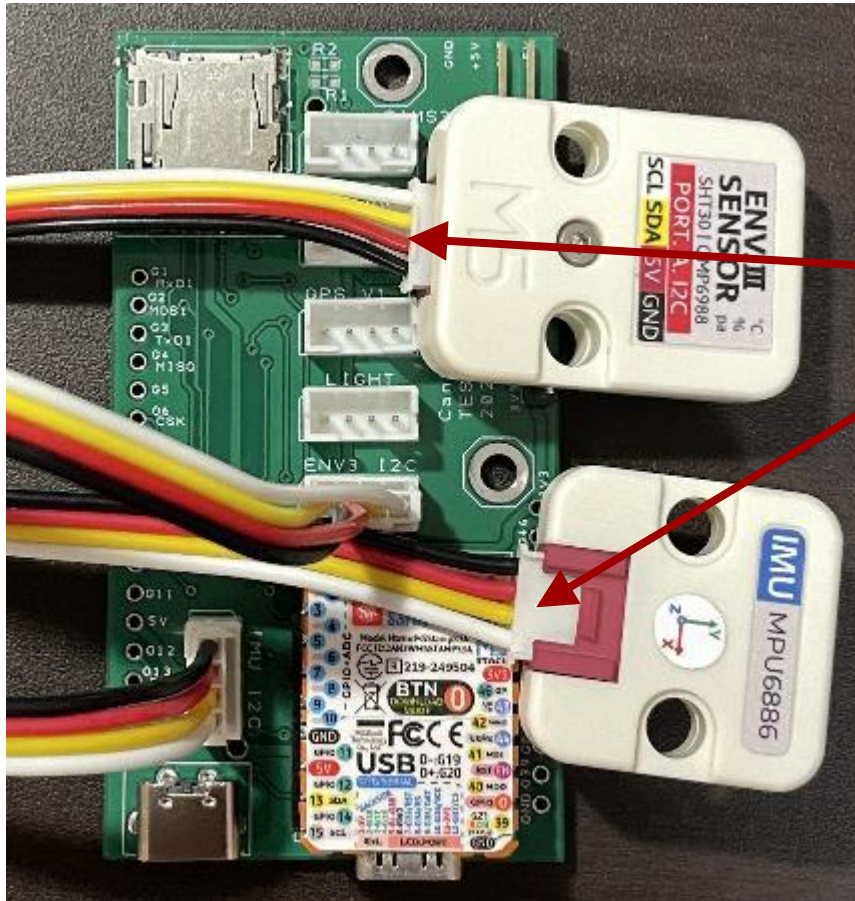


単3電池が3本直列、
約4.5Vの電源になります。

M5 StampS3のLEDが緑点滅し出した
らSDカードに記録されています。

I2C通信

I2Cの接続



ENV III と IMU MPU6886 を Grove コネクタに接続します。
これら Grove コネクタは、I2C 端子です。

白線: SCL
黄線: SDA
赤線: +V
黒線: GND

I2C 接続は 1 つの SDA/SCL に複数接続が可能ですので、2 つ接続しています。

I2C通信

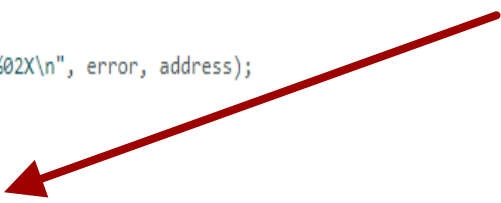
I2Cデバイスの確認

I2C_Scan.ino

```
1  #include "Wire.h"
2
3  void setup() {
4      Serial.begin(115200);
5      Wire.begin();
6  }
7
8  void loop() {
9      byte error, address;
10     int nDevices = 0;
11
12     delay(5000);
13
14     Serial.println("Scanning for I2C devices ...");
15     for(address = 0x01; address < 0x7f; address++){
16         Wire.beginTransmission(address);
17         error = Wire.endTransmission();
18         if (error == 0){
19             Serial.printf("I2C device found at address 0x%02X\n", address);
20             nDevices++;
21         } else if (error != 2){
22             Serial.printf("Error %d at address 0x%02X\n", error, address);
23         }
24     }
25     if (nDevices == 0){
26         Serial.println("No I2C devices found");
27     }
28 }
29
```

「I2C_Scan.ino」を読み込んで、ビルドして書き込んでください。

3つのセンサのアドレスが出れば、
正常に接続されています。



I2C通信

I2Cデバイスの確認

```
16:34:25.287 -> Scanning for I2C devices ...
16:34:25.321 -> I2C device found at address 0x44
16:34:25.321 -> I2C device found at address 0x68
16:34:25.321 -> I2C device found at address 0x70
16:34:30.305 -> Scanning for I2C devices ...
16:34:30.340 -> I2C device found at address 0x44
16:34:30.340 -> I2C device found at address 0x68
16:34:30.340 -> I2C device found at address 0x70
16:34:35.319 -> Scanning for I2C devices ...
16:34:35.353 -> I2C device found at address 0x44
16:34:35.353 -> I2C device found at address 0x68
16:34:35.353 -> I2C device found at address 0x70
```

**0x44が、ENV IIIのSHT30センサです。
SHT30は、温湿度のデータを取得します。**

**0x68が、IMUの MPU6886センサです。
MPU6886は、加速度・ジャイロのデータを取得します。**

**0x70が、ENV IIIのQMP6988センサです。
QMP6988は、気圧のデータを取得します。
高度計算に利用します。**

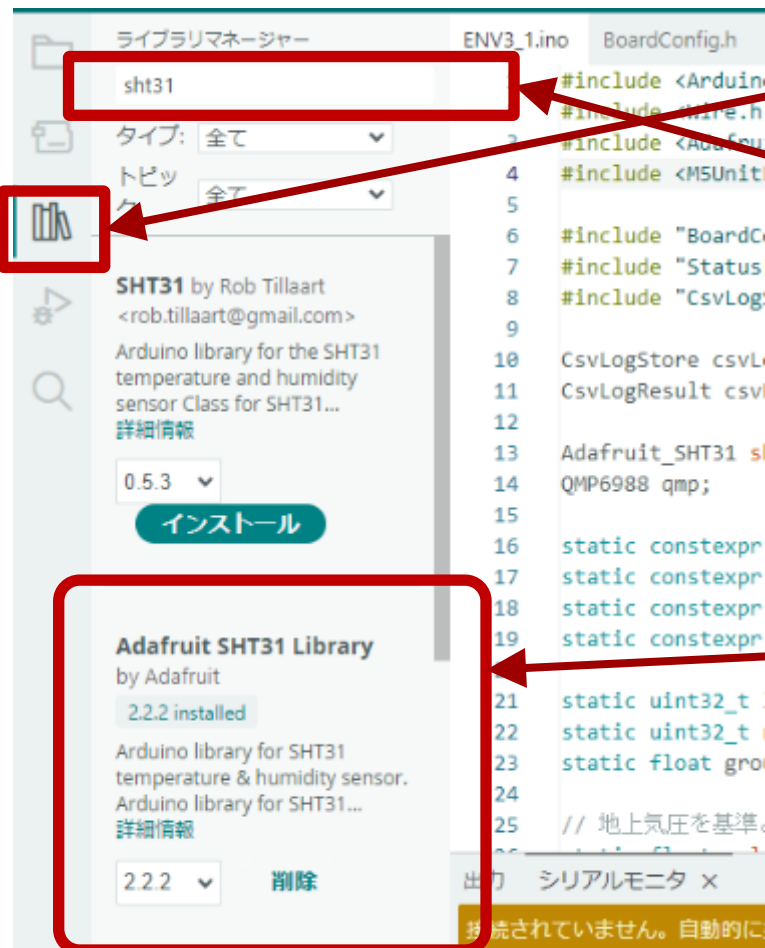
I2C通信

ENV III センサ

温湿度、気圧を測定します。

また、高度も計算してくれます。

「ENV3_1.ino」を開きます。



ライブラリを開きます。

検索に「sht31」と入れます。

Adafruit SHT31 Library by
Adafruit をインストールします。

SHT31ライブラリですが、セン
サはSHT30です。

・ 温度と湿度が測定されます。

I2C通信

ENV III センサ

温湿度、気圧を測定します。

ライブラリを開きます。

検索に「M5UnitENV」と入れます。



M5Unit-ENV by M5Stack をインストールします。
QMP6988センサのライブラリです。
・気圧と温度が測定され、
気圧、温度、高度が計算されます。

ENV III センサ

I2C通信

```

101 void loop() {
102     static char csvLine[CSV_ROW_MAX_LENGTH];
103
104     uint32_t now = millis();
105
106     if (now - lastLogMs < LOG_INTERVAL_MS) {
107         return;
108     }
109
110     lastLogMs += LOG_INTERVAL_MS;
111     rowCount++;
112
113     float shtTemp, shtHumidity;
114     if (!sht31.readBoth(&shtTemp, &shtHumidity)) {
115         stopWithError("SHT31 read failed.");
116     }
117
118     if (!qmp.update()) {
119         stopWithError("QMP6988 update failed.");
120     }
121
122     uint32_t timeMs = millis();
123
124     snprintf(csvLine, sizeof(csvLine), "%lu,%lu,%.2f,%.2f,%.2f,%.2f,%.2f",
125             (unsigned long)timeMs, (unsigned long)rowCount, shtTemp, shtHumidity,
126             qmp.cTemp, qmp.pressure, qmp.altitude - refAltitude); // 海拔高度から基準高度を引いて相対高度に
127
128     csvResult = csvLog.println(csvLine);
129
130     if (!csvResult.ok) {
131         stopWithError("CSV write failed.");
132     }
133
134     if (rowCount % 10 == 0) {
135         Serial.print("CSV row: ");
136         Serial.println(csvLine);
137     }
138 }

```

温度と湿度を取得

気圧と温度と高度の計算

気圧と温度と高度の計算結果を取得

I2C通信

ENV III センサ

自作の高度計算関数も付けました。

```
24
25 // 【参考】地上気圧 refPa を基準とした相対高度 [m] を直接計算する式。
26 // 現在はライブラリの標準メソッド (qmp.altitude) を使うため未使用ですが、
27 // 高度計算の式を学ぶための参考として残しています。
28 // qmp.altitude も同じ形の式 (ただし基準気圧は 101325 Pa 固定) で計算されています。
29 // ※標準メソッド化の改修は Claude Fable 5 (Anthropic の AI) が実施 (2026-07-02)
30 static float calcAltitude(float pressurePa, float tempC, float refPa) {
31     return (powf(refPa / pressurePa, 1.0f / 5.257f) - 1.0f) * (tempC + 273.15f) / 0.0065f;
32 }
33
```

I2C通信

ENV III センサ

温度(°C), 気圧(Pa), 高度(m)などが出力されます。

```
18:54:17.709 -> CSV row: 24727,220,30.62,62.95,30.69,100703.25,-0.21
18:54:18.722 -> CSV row: 25727,230,30.59,62.95,30.69,100704.19,-0.30
18:54:19.705 -> CSV row: 26727,240,30.62,62.95,30.70,100702.81,-0.17
18:54:20.706 -> CSV row: 27727,250,30.61,62.95,30.70,100703.31,-0.22
18:54:21.705 -> CSV row: 28727,260,30.62,62.95,30.70,100702.50,-0.15
18:54:22.722 -> CSV row: 29727,270,30.59,62.91,30.70,100701.50,-0.06
18:54:23.722 -> CSV row: 30727,280,30.61,62.90,30.71,100701.44,-0.05
18:54:24.722 -> CSV row: 31727,290,30.61,62.91,30.71,100700.81,0.00
18:54:25.705 -> CSV row: 32727,300,30.63,62.94,30.71,100700.00,0.08
18:54:26.707 -> CSV row: 33727,310,30.61,62.90,30.71,100700.88,-0.00
18:54:27.705 -> CSV row: 34727,320,30.62,62.90,30.71,100701.31,-0.04
18:54:28.707 -> CSV row: 35727,330,30.62,62.87,30.72,100701.75,-0.08
18:54:29.722 -> CSV row: 36727,340,30.62,62.91,30.72,100701.94,-0.09
18:54:30.705 -> CSV row: 37727,350,30.63,62.87,30.72,100701.75,-0.08
```

```
133
134   if (rowCount % 5 == 0) {
135       //Serial.print("CSV row: ");
136       //Serial.println(csvLine);
137
138       Serial.println(String(shtTemp));
139   }
140
```

CSVの部分をコメントアウトしてください。

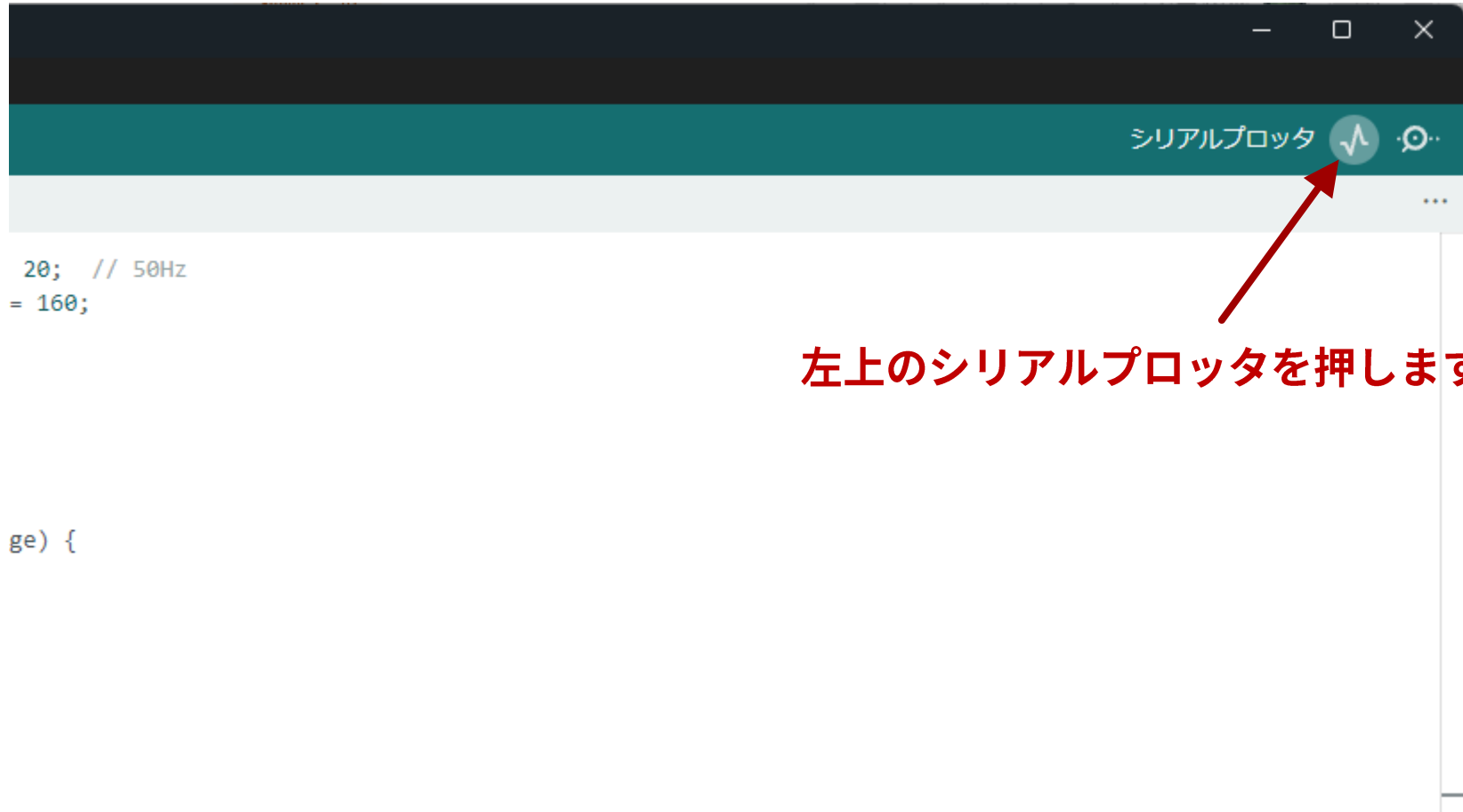
String(shtTemp)の部分のコメントを外してください。

シリアルプロッタで、温度がプロットできます。

I2C通信

ENV III センサ

温度のグラフを書いてみましょう。



左上のシリアルプロッタを押します。

I2C通信

ENV III センサ

温度のグラフを書いてみました。



I2C通信

MPU6886 6軸センサプログラム

「MPU6886.ino」を開きます。



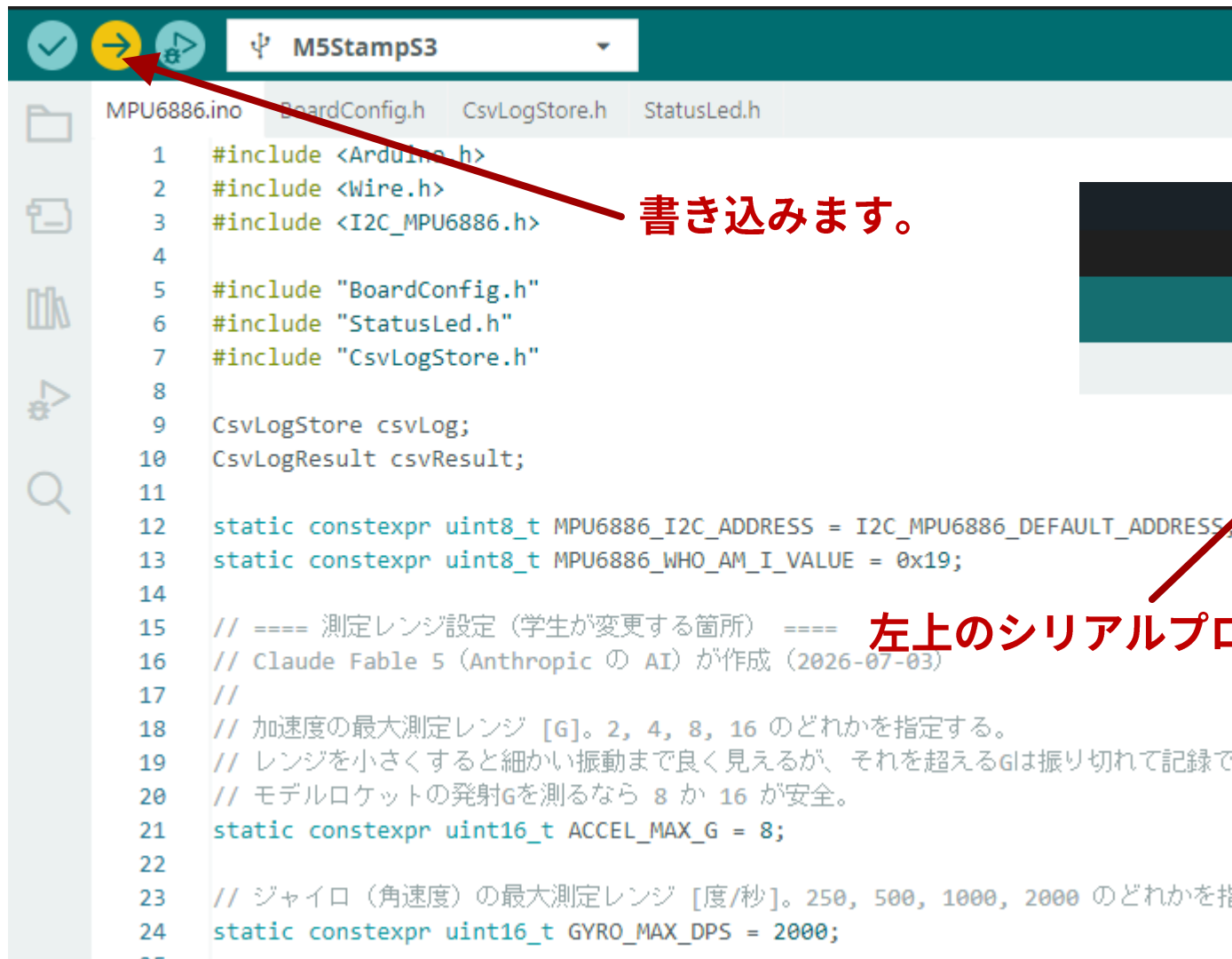
ライブラリを開きます。

検索に「I2C_MPU6886」と入れます。

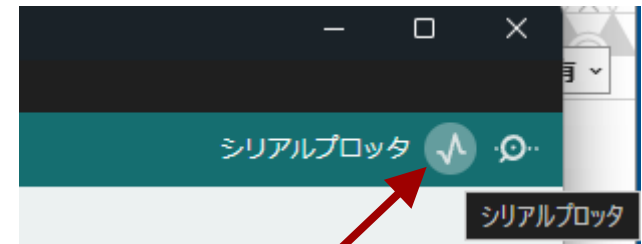
I2C MPU6886 IMU by TANAKA Masayukiをインストールします。

I2C通信

MPU6886 6軸センサプログラム



書き込みます。

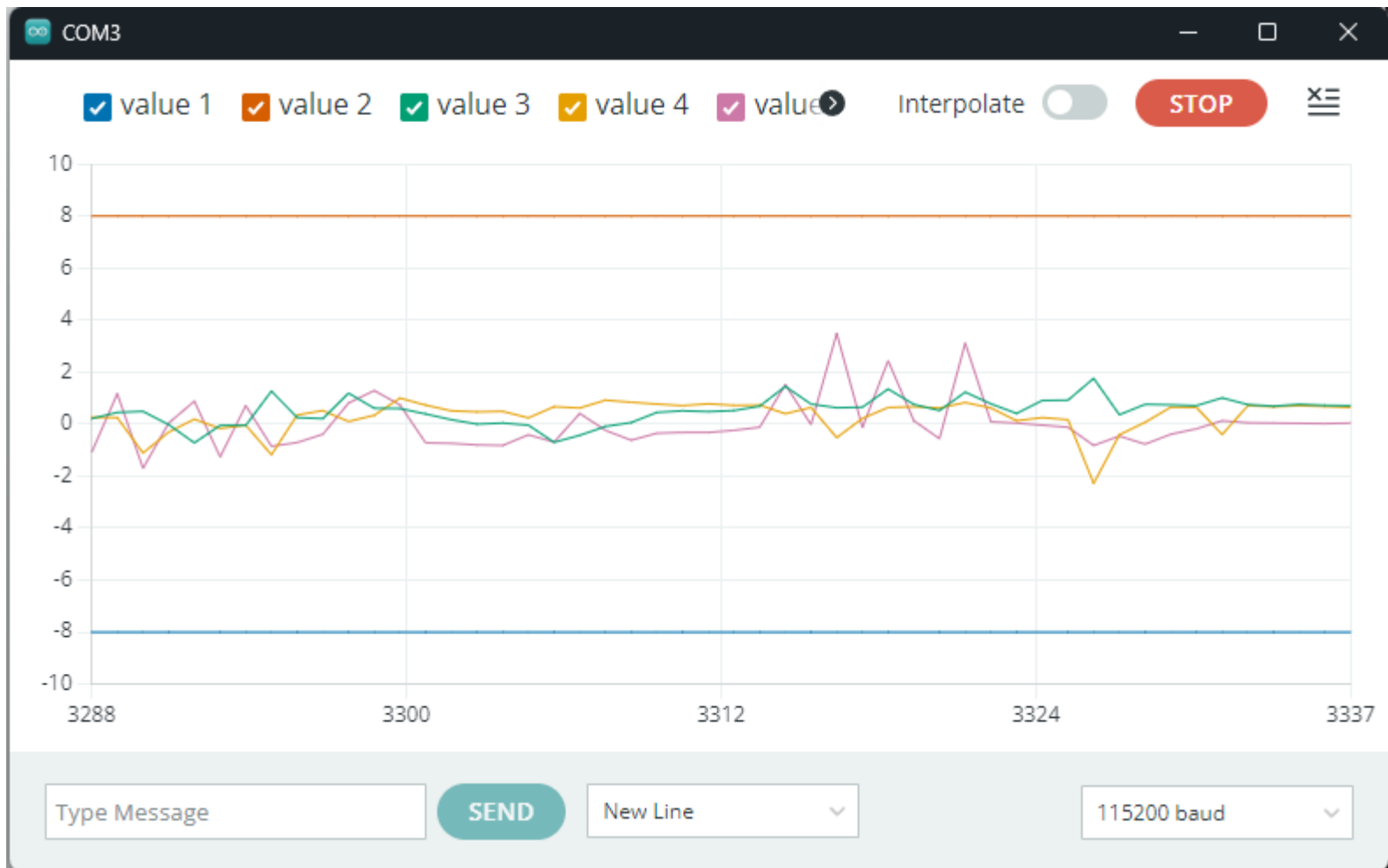


左上のシリアルプロッタを押します。

I2C通信

MPU6886 6軸センサプログラム

3軸の加速度(m/s^2)の波形がでてきます。ボードを揺らしてみてください。



I2C通信

MPU6886 6軸センサプログラム

accelerationをコメントアウトして、gyroscopeのコメントを外します。

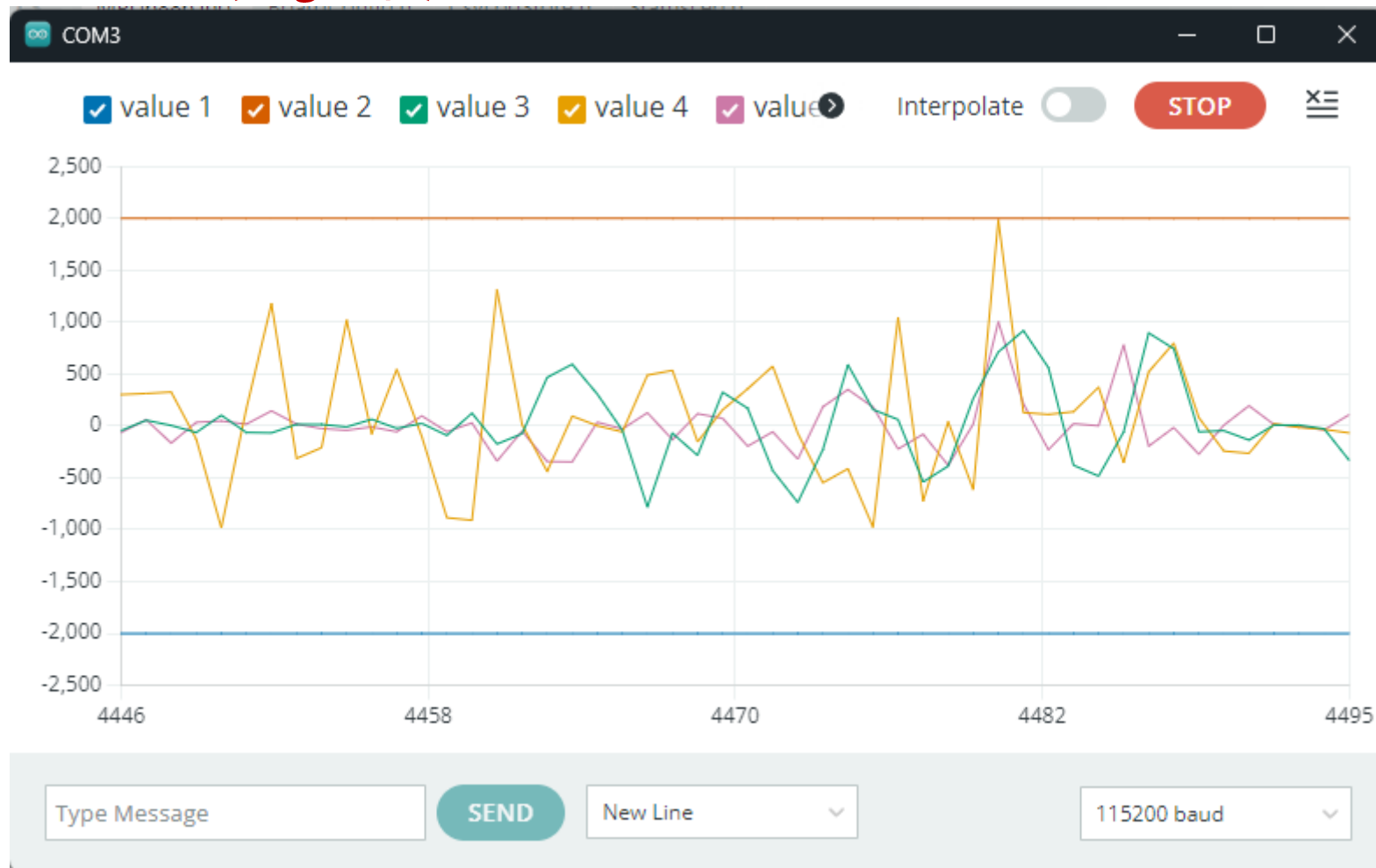
そして、書き込みます。

```
199  if (rowCount % 10 == 0) {
200      //acceleration
201      //Serial.println("-8,8," + String(accelXG) + "," + String(accelYG) + "," + String(accelZG));
202
203      //gyroscope
204      Serial.println("-2000,2000," + String(gyroXDps) + "," + String(gyroYDps) + "," + String(gyroZDps));
205  }
206  }
```

I2C通信

MPU6886 6軸センサプログラム

3軸の角速度(degree/s)の波形がでてきます。ボードを回してみてください。



I2C通信

MPU6886 6軸センサプログラム

加速度と角速度の最大レンジが設定できます。

```
15 // ==== 測定レンジ設定（学生が変更する箇所） ====
16 // Claude Fable 5 (Anthropic の AI) が作成 (2026-07-03)
17 //
18 // 加速度の最大測定レンジ [G]。2, 4, 8, 16 のどれかを指定する。
19 // レンジを小さくすると細かい振動まで良く見えるが、それを超えるGは振り切れて記録できない。
20 // モデルロケットの発射Gを測るなら 8 か 16 が安全。
21 static constexpr uint16_t ACCEL_MAX_G = 8;
22
23 // ジャイロ（角速度）の最大測定レンジ [度/秒]。250, 500, 1000, 2000 のどれかを指定する。
24 static constexpr uint16_t GYRO_MAX_DPS = 2000;
25
```

加速度と角速度と温度を取得します。

```
158 uint32_t timeMs = millis();
159 float accelXG = 0.0f;
160 float accelYG = 0.0f;
161 float accelZG = 0.0f;
162 float gyroXDps = 0.0f;
163 float gyroYDps = 0.0f;
164 float gyroZDps = 0.0f;
165 float temperatureC = 0.0f;
166
167 imu.getAccel(&accelXG, &accelYG, &accelZG);
168 imu.getGyro(&gyroXDps, &gyroYDps, &gyroZDps);
169 imu.getTemp(&temperatureC);
170
```

加速度を取得します。

角速度を取得します。

温度を取得します。

I2C通信

SDカードには、以下のように保存されています。

	A	B	C	D	E	F	G	H	I
1	time_ms	row	accel_x_g	accel_y_g	accel_z_g	gyro_x_dps	gyro_y_dps	gyro_z_dps	temperature_c
2	1713	1	0.17749	0.018799	1.04248	4.821777	5.126953	-4.272461	39.88
3	1733	2	0.177002	0.010498	1.040527	4.638672	5.432129	-4.02832	39.62
4	1753	3	0.17627	0.017822	1.043457	4.150391	4.333496	-4.02832	40.1
5	1773	4	0.177979	0.013672	1.049316	3.967285	4.699707	-4.272461	40.25
6	1793	5	0.174316	0.016846	1.04248	4.02832	4.943848	-3.90625	39.76
7	1813	6	0.178711	0.02002	1.04248	4.211426	4.699707	-3.845215	39.01
8	1833	7	0.176025	0.01416	1.044678	3.601074	5.249023	-4.02832	39.43
9	1853	8	0.181396	0.019775	1.042725	4.760742	5.065918	-4.272461	40.14
10	1873	9	0.175537	0.013184	1.04248	4.394531	5.004883	-4.089355	39.42
11	1893	10	0.17749	0.011719	1.046143	4.089355	5.554199	-3.417969	38.67
12	1913	11	0.179932	0.011719	1.044434	4.516602	5.004883	-4.02832	39.16
13	1933	12	0.176758	0.013672	1.041016	5.310059	5.065918	-4.455566	39.44
14	1953	13	0.18042	0.017822	1.036621	3.78418	4.577637	-3.78418	39.73
15	1973	14	0.178711	0.016846	1.04126	3.967285	4.882812	-3.845215	39.5
16	1993	15	0.176514	0.019531	1.041992	4.394531	4.882812	-3.78418	39.41
17	2013	16	0.175293	0.013916	1.040527	5.004883	5.371094	-4.455566	39.8
18	2033	17	0.17749	0.009766	1.043213	4.272461	5.249023	-4.02832	39.52
19	2053	18	0.181885	0.014404	1.04248	4.638672	5.126953	-3.967285	39.91
20	2073	19	0.177734	0.020508	1.040039	4.150391	5.554199	-4.02832	39.85

I2C通信

ENV IIIとMPU6886をまとめました。
SDカードに書き込んでみましょう。
「ENV3_MPU6886.ino」をM5StampS3Aに書き込みます。

SDカードにCSV形式で書きこまれます。

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	time_ms	row	sht_temperature_c	humidity_rh	qmp_temperature_c	pressure_pa	altitude_m	accel_x_g	accel_y_g	accel_z_g	gyro_x_dps	gyro_y_dps	gyro_z_dps	imu_temperature_c
2	2900	1	28.52	58.14	28.44	101012.5	0.02	-0.013916	-0.012451	1.065186	3.662109	5.065918	-3.601074	36.09
3	2950	2	28.54	58.17	28.44	101012.44	0.03	-0.01709	-0.013672	1.062988	3.601074	4.577637	-3.967285	38.11
4	3000	3	28.55	58.15	28.44	101011.56	0.11	-0.017334	-0.009277	1.061523	3.051758	4.394531	-3.356934	36.9
5	3050	4	28.54	58.16	28.45	101011.31	0.13	-0.01709	-0.010254	1.063477	4.333496	4.943848	-4.150391	38.4
6	3100	5	28.55	58.16	28.45	101011.56	0.11	-0.014648	-0.009766	1.062256	3.479004	4.455566	-3.479004	37.43
7	3150	6	28.52	58.14	28.45	101011.5	0.11	-0.014893	-0.010254	1.062012	3.662109	4.394531	-3.90625	36.29
8	3200	7	28.54	58.17	28.45	101011	0.16	-0.020996	-0.013916	1.06665	3.845215	4.211426	-3.234863	36.66
9	3250	8	28.52	58.18	28.45	101010.5	0.2	-0.015137	-0.011719	1.059814	3.78418	5.187988	-4.089355	37.59
10	3300	9	28.55	58.12	28.45	101011.12	0.14	-0.015625	-0.01001	1.0625	3.662109	5.187988	-3.967285	37.29
11	3350	10	28.55	58.13	28.46	101010.88	0.17	-0.017334	-0.008545	1.064697	3.295898	4.577637	-3.90625	37.7
12	3400	11	28.55	58.19	28.45	101011.25	0.13	-0.011475	-0.009033	1.067871	3.845215	4.943848	-3.662109	37.37
13	3450	12	28.57	58.16	28.46	101012.06	0.06	-0.01709	-0.011475	1.059814	2.624512	3.723145	-3.540039	36.63
14	3500	13	28.57	58.16	28.46	101012.94	-0.01	-0.017578	-0.016602	1.061279	3.78418	4.272461	-3.601074	36.02
15	3550	14	28.57	58.2	28.46	101012.81	0	-0.016357	-0.00708	1.060791	3.845215	5.371094	-3.90625	37.85
16	3600	15	28.55	58.16	28.46	101012.62	0.01	-0.020508	-0.011963	1.066895	3.78418	4.699707	-3.479004	36.51
17	3650	16	28.54	58.16	28.46	101011.94	0.07	-0.017578	-0.006592	1.062012	4.02832	4.638672	-3.78418	37.47
18	3700	17	28.57	58.18	28.47	101010.88	0.17	-0.014893	-0.006836	1.0625	3.479004	4.455566	-4.455566	37.62
19	3750	18	28.57	58.16	28.47	101012.25	0.05	-0.017578	-0.01123	1.066162	4.272461	5.249023	-3.90625	37.52
20	3800	19	28.55	58.12	28.47	101012.25	0.05	-0.015625	-0.012939	1.063232	3.601074	5.249023	-4.150391	35.89
21	3850	20	28.57	58.16	28.47	101011.12	0.15	-0.015625	-0.008301	1.056641	4.02832	4.577637	-4.150391	36.93
22	3900	21	28.54	58.17	28.47	101012.06	0.06	-0.019042	-0.010096	1.067871	3.00625	4.921777	-4.02832	36.22

光センサUNIT LIGHT [U021]

G5番をアナログ入力に、G7番をデジタル入力に使用します。

LIGHT_AD5_IO7_Sample.ino

```
1  #include <Arduino.h>
2
3  #define A_OUT 5
4  #define D_OUT 7
5
6  void setup() {
7    Serial.begin(115200);
8    delay(1500);
9
10   pinMode(A_OUT, INPUT);
11   pinMode(D_OUT, INPUT);
12
13   Serial.println("Stamp-S3 LIGHT Demo Sample");
14 }
15
16 void loop() {
17   float vInput = 3.3 * analogRead(A_OUT) / 4095; //電圧Vに変換
18   int dInput = digitalRead(D_OUT);
19
20   String log = "";
21   log += String(vInput) + "," + String(dInput);
22   log += ",3.5,0";
23   Serial.println(log);
24
25   delay(100);
26 }
```

光センサは、

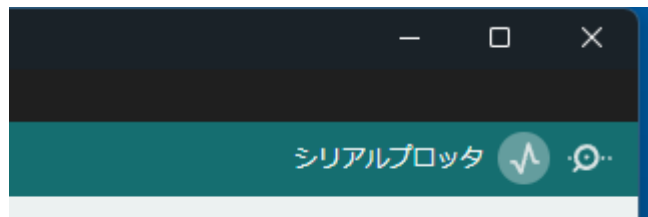
- ・ 暗いほど大きい値です。
- ・ デジタルも暗いと 1 になります。
- ・ 明るいと 0 になります。

アナログ値は 3.3Vが、
0～4095 の範囲で出力されます。

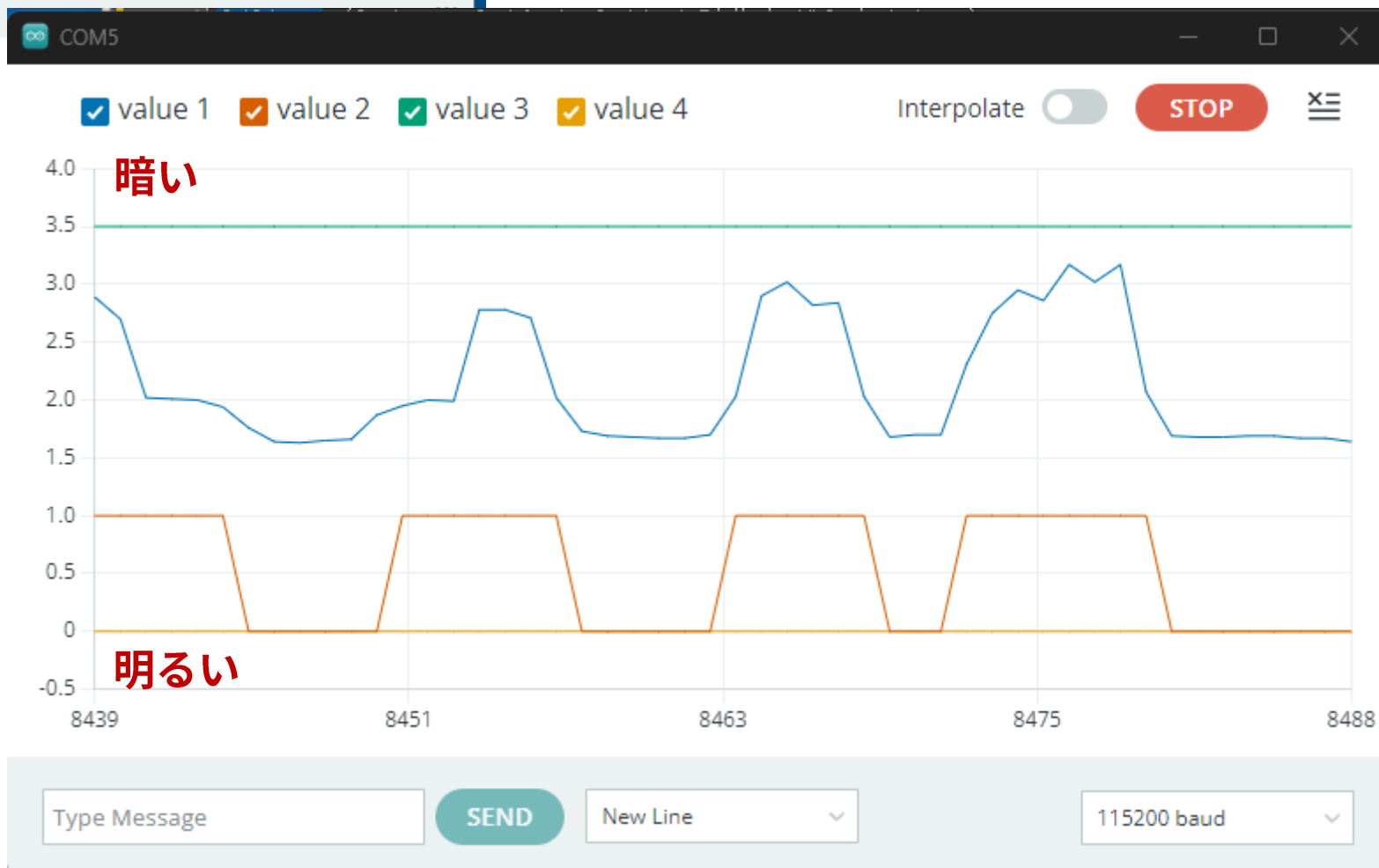
デジタル値は、
0 と 1 の範囲で出力されます。

可変抵抗を左に回すと、感度が
上がります。

光センサUNIT LIGHT [U021]



画面左上のシリアルプロッタを開きます。



SDカード保存

ENV IIIとMPU6886とLIGHTをまとめました。

SDカードに書き込んでみましょう。

「ENV3_MPU6886_LIGHT.ino」をM5StampS3Aに書き込みます。

SDカードにCSV形式で書きこまれます。

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	time_ms	row	sht_temp	humidity	qmp_temp	pressure	altitude_m	accel_x_g	accel_y_g	accel_z_g	gyro_x_dp	gyro_y_dp	gyro_z_dp	imu_temp	light_v	light_d
2	3057	1	29.2	59.24	29.17	101027.9	0.08	0.340576	0.15918	1.1875	-659.546	-280.212	-31.9824	37.07	1.69	0
3	3107	2	29.18	59.2	29.18	101028.9	0	0.758057	0.720215	2.230713	12.81738	172.4854	118.4692	36.89	1.66	0
4	3157	3	29.17	59.25	29.18	101029.9	-0.09	0.643311	0.272217	0.2146	616.8823	-210.815	32.4707	37.53	1.66	0
5	3207	4	29.2	59.19	29.18	101029.8	-0.08	-0.23315	0.495117	-0.57105	96.74072	17.21191	-80.7495	36.6	1.64	0
6	3257	5	29.17	59.2	29.18	101030.1	-0.1	-0.2207	0.157227	0.839111	-677.002	-267.09	-11.2915	36.86	1.66	0
7	3307	6	29.18	59.24	29.18	101029.9	-0.09	0.681396	0.751953	2.326172	-198.547	229.3091	159.79	37.4	1.65	0
8	3357	7	29.21	59.22	29.18	101028.4	0.04	1.104492	0.198975	0.842529	601.7456	-213.44	6.713867	36.71	1.63	0
9	3407	8	29.2	59.28	29.19	101028.6	0.03	-0.27979	0.493896	-0.54346	239.8071	144.2871	-25.8179	36.33	1.67	0
10	3457	9	29.18	59.31	29.18	101029.1	-0.02	-0.31079	0.18457	0.630859	-591.492	-332.336	60.48584	36.6	1.66	0
11	3507	10	29.21	59.29	29.18	101027.7	0.11	0.568359	0.472168	2.227051	-381.592	287.7197	113.9526	37.2	1.65	0
12	3557	11	29.2	59.3	29.19	101030.8	-0.16	1.609863	0.140869	1.384766	616.272	275.0854	234.375	36.37	1.67	0
13	3607	12	29.2	59.33	29.19	101029.8	-0.08	-0.66968	0.26416	-1.01172	550.2319	-342.59	-79.5898	36.8	1.67	0
14	3657	13	29.21	59.34	29.19	101028.4	0.05	-0.78589	0.209961	-0.20947	-568.97	-348.938	14.89258	36.8	1.66	0
15	3707	14	29.2	59.3	29.19	101028.7	0.02	0.433594	0.042236	1.863281	-697.388	237.7319	-53.1006	35.86	1.64	0
16	3757	15	29.2	59.34	29.19	101028.5	0.04	0.355957	0.495605	2.39502	138.4277	218.1396	234.375	37.3	1.66	0
17	3807	16	29.2	59.36	29.2	101029.6	-0.06	1.419678	0.039551	0.866943	710.7544	-111.145	71.77734	36.14	1.65	0
18	3857	17	29.23	59.41	29.2	101028.8	0.01	-0.59912	0.45459	-0.54126	212.1582	177.1851	-56.5186	37	1.66	0

GPS受信

GPS V1.1 (AT6668 + MAX2659)



衛星から電波を受信できると、
青いLEDが点滅します。

受信データは、\$から始まるヘッダ付きで、
受信されます。

データの詳細は、各自調べてください。

メッセージ (COM5) の莫大のモジュールにメッセージを送信するにはEnter

```
14:19:42.270 -> $GPRMC,01,01,01,A,14.1942,N,101.1942,E,0.0,0.0,0.0,0.0,0.0,0.0  
14:19:42.593 -> ,00,00*76  
14:19:42.703 -> $GPTXT,01,01,01,ANTENNA OPEN*25  
14:19:43.003 -> $GNRMC,051942.00,A,3406.12508,N,13507.08271,E,0.28,,050625,,,A,V*22  
14:19:43.703 -> $GNVTG,,,,,0.28,N,0.51,K,A*2A  
14:19:44.033 -> $GNGGA,051942.00,3405.06,2025,00,00*75  
14:19:44.403 -> $GPTXT,01,01,01,ANTENNA OPEN*25  
14:19:44.743 -> ,00,00*74  
14:19:44.853 -> $GPTXT,01,01,01,ANTENNA OPEN*25  
14:19:45.183 -> $GNRMC,051944.00,A,3406.12508,N,13507.08271,E,0.17,,050625,,,A,V*28  
14:19:45.888 -> $GNVTG,,,,,0.17,N,0.31,K,A*20  
14:19:46.183 -> $GNGGA,051944.00,340,,0.15,N,0.27,K,A*25  
14:19:46.633 -> $GNGGA,051945.00,3406.12508,N,13507.08271,E,1.08,1.4,7.87,M,38.05,M,,*76  
14:19:47.373 -> $GNGSA,A,3,06,09,11,19,01,ANTENNA OPEN*25  
14:19:47.813 -> $GNRMC,051947.00,A,3406.12508,N,13507.08271,E,0.32,,050625,,
```

GPS受信

GPS V1.1 (AT6668 + MAX2659)

「GPSV1.1_Sample2.ino」を開きます。

便利なGNSSデコードライブラリがありました。

ライブラリを開きます。

検索に「TinyGPSPlus」と入力します。

TinyGPSPlus-ESP32 by Mikal Hart をインストールします。

The screenshot shows the Arduino IDE Library Manager interface. On the left, the search bar contains 'TinyGPSPlus'. Below it, the results list shows 'TinyGPSPlus by Mikal Hart' with version 1.0.3 and an 'インストール' (Install) button. Below that, 'TinyGPSPlus-ESP32 by Mikal Hart' is listed with version 0.0.2 and a '削除' (Remove) button. A red box highlights the 'TinyGPSPlus-ESP32 by Mikal Hart' entry. A red arrow points from the search bar to the 'TinyGPSPlus' entry, and another red arrow points from the 'TinyGPSPlus-ESP32 by Mikal Hart' entry to the code editor. The code editor on the right shows the 'GPSV1.1_Sample2.ino' file with the following code:

```
1 #include <Arduino.h>
2 #include <TinyGPSPlus.h>
3
4 // The TinyGPSPlus object
5 TinyGPSPlus gps;
6 unsigned long lastRead = 0;
7
8 void setup() {
9   Serial.begin(115200);
10  Serial2.begin(115200, SERIAL_8N1, 8);
11  delay(1500);
12
13  Serial.println(F("GPS V1.1 Sam
14
15
16 void loop() {
17   if (Serial2.available() > 0) {
18     gps.encode(Serial2.read());
19   }
20   unsigned long now = millis();
21   if (now - lastRead > 1000) //
22   {
23     lastRead = now;
24
25     if (gps.location.isValid() &
26         int utcHour = gps.time.hou
27         int utcMinute = gps.time.m
28         int utcSecond = gps.time.s
```

GPS受信

GPS V1.1 (AT6668 + MAX2659)

「GPSV1.1_Sample2.ino」を開きます。

GNSSのデータを分かりやすく表示させました。

```
15:03:38.879 -> ----- GNSS Data -----  
15:03:38.879 -> UTC: 2025/6/5UTC: 2025/6/5 6:3:35  
15:03:38.879 -> JST: 2025/6/5 15:3:35  
15:03:38.879 -> Latitude: 34.102098, Longitude: 135.118062  
15:03:38.879 -> Speed (km/h): 0.98  
15:03:38.879 -> Course (deg): 193.63  
15:03:38.879 -> Satellites: 14  
15:03:38.879 -> Altitude (m): 9.13  
15:03:38.879 -> HDOP: 90
```

GPS受信

GPS V1.1 (AT6668 + MAX2659)

「GPSV1.1_Sample3.ino」を開きます。
このサンプルは、緯度と経度だけを取得するサンプルです。

```

16 void loop() {
17   if (Serial2.available() > 0) {
18     gps.encode(Serial2.read());
19   }
20   unsigned long now = millis();
21   if (now - lastRead > 250) // 250ms更新
22   {
23     lastRead = now;
24
25     if (gps.location.isValid()) {
26
27       String log = "";
28       log += String(gps.location.lat(), 6) + "," + String(gps.location.lng(), 6);
29       Serial.println(log);
30     }
31   }
32
33   delay(10);
34 }
--

```

SDカードに書き出します。

	A	B	C	D	E
1	1609	1	34.10207	135.118	
2	1629	2	34.10207	135.118	
3	1649	3	34.10207	135.118	
4	1669	4	34.10207	135.118	
5	1689	5	34.10207	135.118	
6	1709	6	34.10207	135.118	
7	1729	7	34.10207	135.118	
8	1749	8	34.10207	135.118	
9	1769	9	34.10207	135.118	
10	1789	10	34.10207	135.118	
11	1809	11	34.10207	135.118	

**SDカードにすべてのデータを
保存するプログラムを
作ってみましょう。**

全部保存

「ENV3_MPU6886_LIGHT_GPS.ino」があります。

SDカードは、以下が出力されます。

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
1	time_ms	row	sht_temp	humidity	qmp_temp	pressure	altitude_m	accel_x_g	accel_y_g	accel_z_g	gyro_x_dp	gyro_y_dp	gyro_z_dp	imu_temp	light_v	light_d	gps_valid	gps_lat	gps_lng	gps_satellites	
2	2947	1	27.96	58.88	27.89	101049.8	0.08	0.267334	-0.9502	-0.0127	3.78418	5.004883	-3.23486	37.37	3.3	1	1	34.10187	135.1181	10	
3	2997	2	27.97	58.94	27.9	101050.3	0.04	0.268799	-0.9519	-0.00781	3.479004	4.516602	-3.2959	35.56	3.3	1	1	34.10187	135.1181	10	
4	3047	3	27.99	58.91	27.9	101049.3	0.13	0.27002	-0.95166	-0.01001	4.760742	4.821777	-3.78418	37.35	3.3	1	1	34.10187	135.1181	10	
5	3097	4	27.97	58.91	27.9	101050.4	0.03	0.268555	-0.9519	-0.01172	4.516602	6.286621	-4.02832	35.26	3.3	1	1	34.10187	135.1181	10	
6	3147	5	27.99	58.91	27.91	101050.8	0	0.271729	-0.95288	-0.0083	3.845215	6.591797	-3.54004	35.09	3.3	1	1	34.10187	135.1181	10	
7	3197	6	27.97	58.93	27.91	101049.9	0.07	0.265869	-0.95483	-0.00635	4.211426	5.92041	-3.96729	35.47	3.3	1	1	34.10187	135.1181	10	
8	3247	7	27.99	58.92	27.91	101049.3	0.13	0.269775	-0.95288	-0.01172	4.394531	5.981445	-3.72315	36.19	3.3	1	1	34.10187	135.1181	10	
9	3297	8	27.96	58.92	27.91	101049.9	0.07	0.270508	-0.95386	-0.00684	3.662109	4.821777	-3.41797	36.09	3.3	1	1	34.10187	135.1181	10	
10	3347	9	27.99	58.94	27.91	101049.9	0.08	0.274414	-0.94751	-0.0083	3.417969	5.065918	-3.54004	36.67	3.3	1	1	34.10187	135.1181	10	
11	3397	10	28	58.9	27.91	101049.9	0.08	0.262207	-0.95776	-0.01074	3.051758	5.249023	-3.60107	36.2	3.3	1	1	34.10187	135.1181	10	
12	3447	11	27.99	58.92	27.91	101050.4	0.03	0.268799	-0.94995	-0.00855	3.662109	5.615234	-3.60107	36.78	3.3	1	1	34.10187	135.1181	10	
13	3497	12	27.99	58.91	27.91	101049.8	0.08	0.269043	-0.95337	-0.00586	3.78418	5.249023	-3.78418	35.96	3.3	1	1	34.10187	135.1181	10	
14	3547	13	28	58.92	27.91	101049.9	0.07	0.273438	-0.94971	-0.00781	3.601074	5.065918	-3.54004	37.09	3.3	1	1	34.10187	135.1181	10	
15	3597	14	28	58.94	27.92	101049.8	0.08	0.27002	-0.95483	-0.00635	3.601074	6.286621	-3.54004	35.56	3.3	1	1	34.10187	135.1181	10	
16	3647	15	28.02	58.89	27.92	101050.6	0.02	0.269287	-0.95215	-0.00635	3.417969	4.882812	-3.66211	35.64	3.3	1	1	34.10187	135.1181	10	
17	3697	16	28	58.94	27.92	101049.2	0.14	0.274658	-0.95044	-0.00684	4.333496	4.577637	-3.60107	36.39	3.3	1	1	34.10187	135.1181	11	
18	3747	17	27.99	58.94	27.92	101048.6	0.19	0.268799	-0.95068	-0.01367	3.90625	3.479004	-4.39453	36.08	3.3	1	1	34.10187	135.1181	11	
19	3797	18	28	58.95	27.92	101048.5	0.2	0.270264	-0.94946	-0.01514	3.417969	2.868652	-3.41797	36.62	3.3	1	1	34.10187	135.1181	11	
20	3847	19	28	58.95	27.92	101048.8	0.18	0.277832	-0.95142	-0.01123	3.78418	4.577637	-3.479	36.15	3.3	1	1	34.10187	135.1181	11	
21	3897	20	28.02	58.94	27.92	101049.3	0.13	0.270996	-0.95044	-0.01221	3.356934	5.187988	-3.41797	36.72	3.3	1	1	34.10187	135.1181	11	

T9小型W-iFiカメラ

https://www.amazon.co.jp/dp/B0H28912RS?ref=ppx_yo2ov_dt_b_fed_asin_title



この向きにSDカードを入れます。

取説を読んで使ってください。

- ・2時間充電する必要があります。

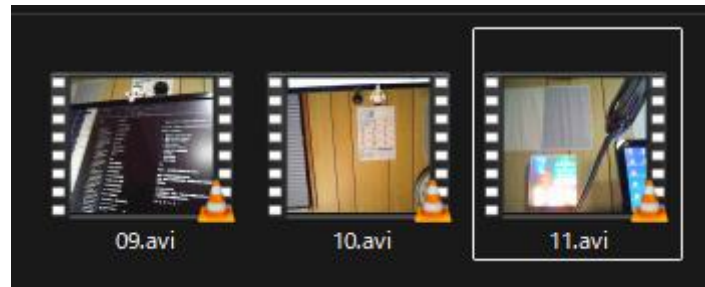
T9小型W-iFiカメラ

こちらが上。



電源スイッチを軽く一回押すと、青色LEDが点滅を始めます。

これで、動画が撮影されています。



本番に向けて、頑張ってください。

ご清聴ありがとうございました。

去年のカメラプログラムの再掲

カメラプログラム

小型カメラ、M5Stack用CamS3-5MP Wi-Fiカメラは、
ArduinoIDEでプログラミングできるカメラモジュールです。



今回は、空撮用に使用します。
WiFiやBluetooth機能は、
使っていません。



カメラプログラム



M5Stack CamS3-5MPは、

- ・ UARTシリアル通信
- ・ SDカード読み書き
- ・ I/Oピン: G0,G19,G20,G44,G43
- ・ グローブ端子
- ・ LED
- ・ カメラ
- ・ マイク

が付いています。

UART:Serial2(RXD:44, TDX:43)

電源は5Vです。

GND

カメラプログラム

M5CamS3Unit_5MPフォルダの「JpegCaptures_5MP.ino」を開いてください。

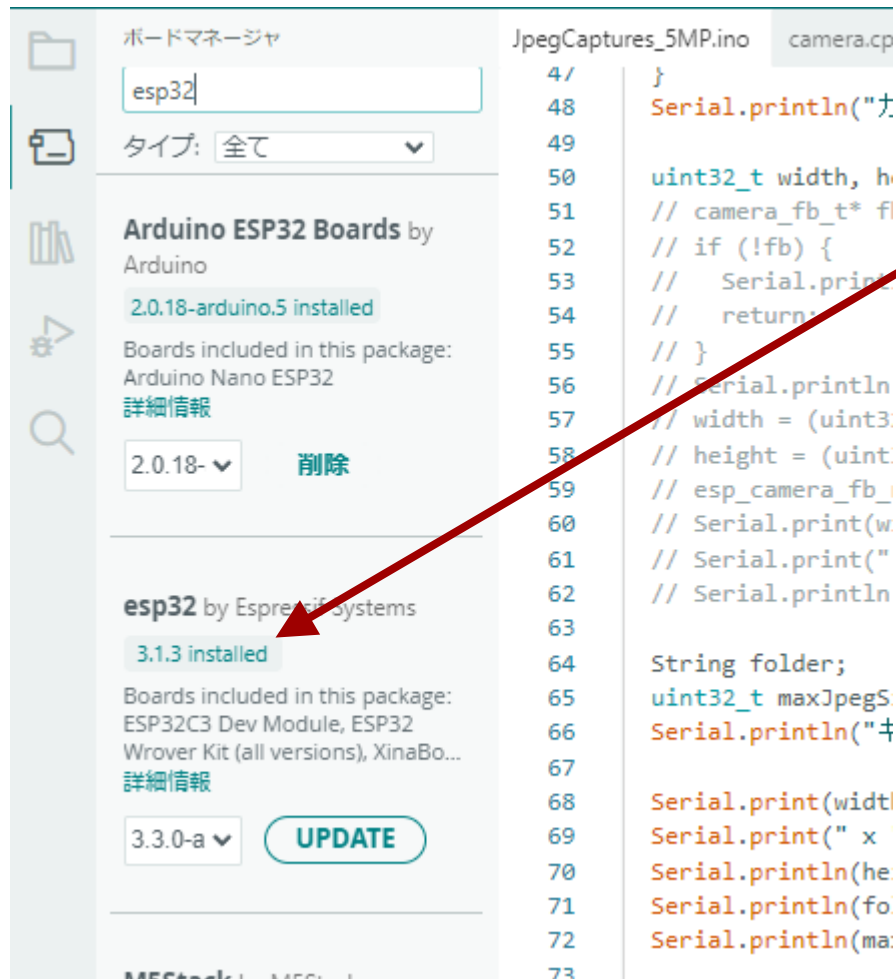
CamS3で検索します。

ボードCOMポートを選択します。



M5UnitCAMS3 -esp32を選択

カメラプログラム

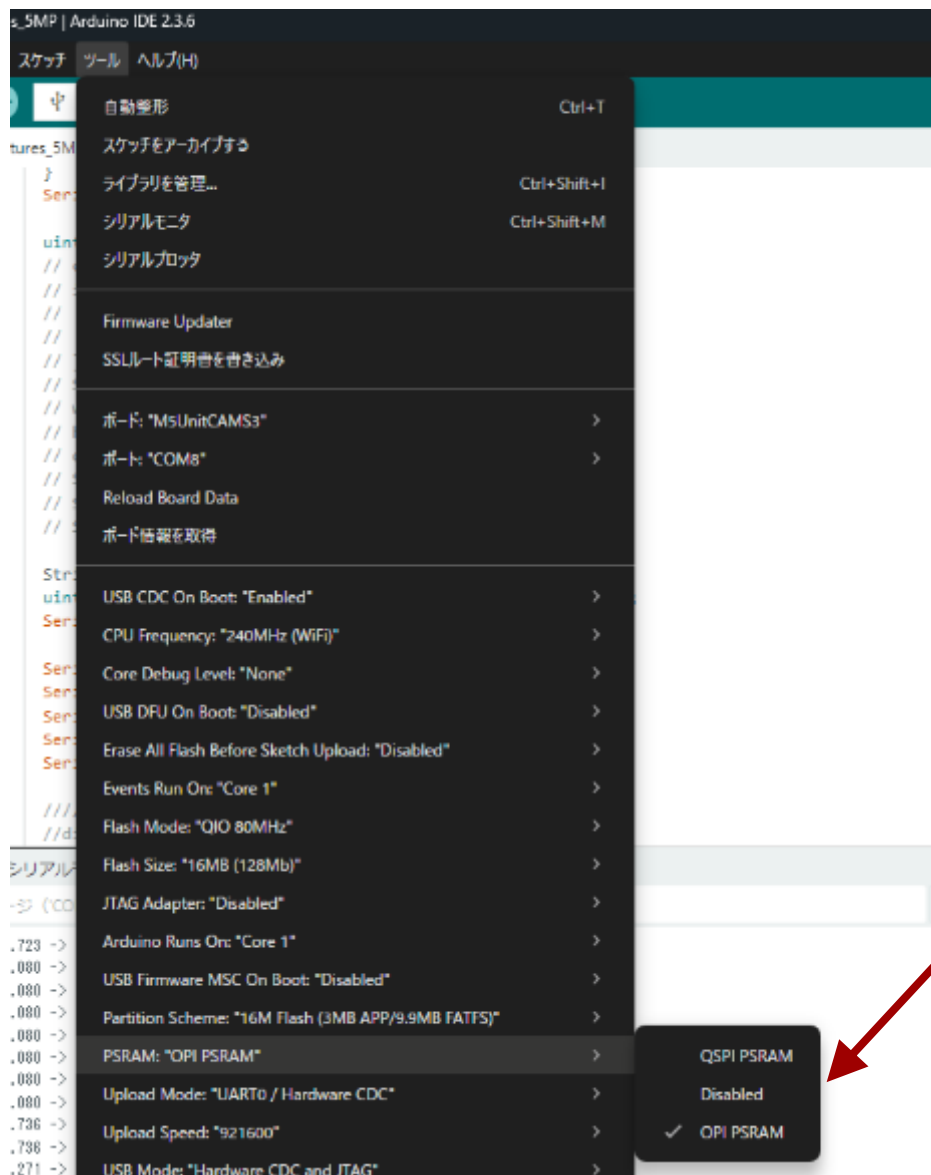


ボードマネージャで、
esp32のバージョンが、
3.1.3 がインストールされている
ことを確認

最新版は使えません。

事前資料「M5CamS3Unit-5MP
の開発環境設定.pdf」にある設定
を実行済みであること。

カメラプログラム



PSRAMが、「OPI PSRAM」になっていることを確認

カメラプログラム

JpegCaptures_5MP.inoを書き込んでください。

10秒ほどで、撮影が終わり、青色LEDが点滅します。

```
01:01:23.535 -> カメラの初期化をします
01:01:23.847 -> □[0;31mE (1224) camera: camera probe ok□[0m
01:01:23.847 -> □[0;31mE (1225) camera: Setting frame size to 320x240□[0m
01:01:23.847 -> カメラが初期化されました
01:01:23.847 -> カメラセンサーを取得します
01:01:23.847 -> PIXFORMAT_JPEGです
01:01:23.847 -> ホワイトバランスのモード設定
01:01:23.847 -> 4枚を空撮影、ホワイトバランスを安定されるため
01:01:24.521 -> カメラを初期化しました
01:01:24.521 -> フォルダ番号最大値検索
01:01:24.647 -> キャプチャ開始
01:01:34.720 -> キャプチャ終了
01:01:34.720 -> 320 x 240
01:01:34.720 -> /000003
01:01:34.720 -> 41290
```

000003フォルダにJpegファイル
が保存されました。

「失敗しました」と出たときは、USBケーブルを抜き差しし、再起動させてください。

カメラプログラム

名前	種類	サイズ
 frame0.jpg	IrfanView JPG File	40 KB
 frame1.jpg	IrfanView JPG File	40 KB
 frame2.jpg	IrfanView JPG File	40 KB
 frame3.jpg	IrfanView JPG File	40 KB
 frame4.jpg	IrfanView JPG File	40 KB
 frame5.jpg	IrfanView JPG File	40 KB
 frame6.jpg	IrfanView JPG File	40 KB
 frame7.jpg	IrfanView JPG File	40 KB
 frame8.jpg	IrfanView JPG File	40 KB
 frame9.jpg	IrfanView JPG File	40 KB
 frame10.jpg	IrfanView JPG File	40 KB
 frame11.jpg	IrfanView JPG File	40 KB
 frame12.jpg	IrfanView JPG File	40 KB
 frame13.jpg	IrfanView JPG File	40 KB
 frame14.jpg	IrfanView JPG File	40 KB
 frame15.jpg	IrfanView JPG File	40 KB
 frame16.jpg	IrfanView JPG File	41 KB
 frame17.jpg	IrfanView JPG File	41 KB
 frame18.jpg	IrfanView JPG File	41 KB
 frame19.jpg	IrfanView JPG File	40 KB
 frame20.jpg	IrfanView JPG File	41 KB

Jpegファイル50枚、保存されています。

この画像は、
・ 5FPSで撮影されています。
1秒間に5フレーム

このJpegファイルから動画を作成します。

FFmpegで動画作成

FFmpegをダウンロードします

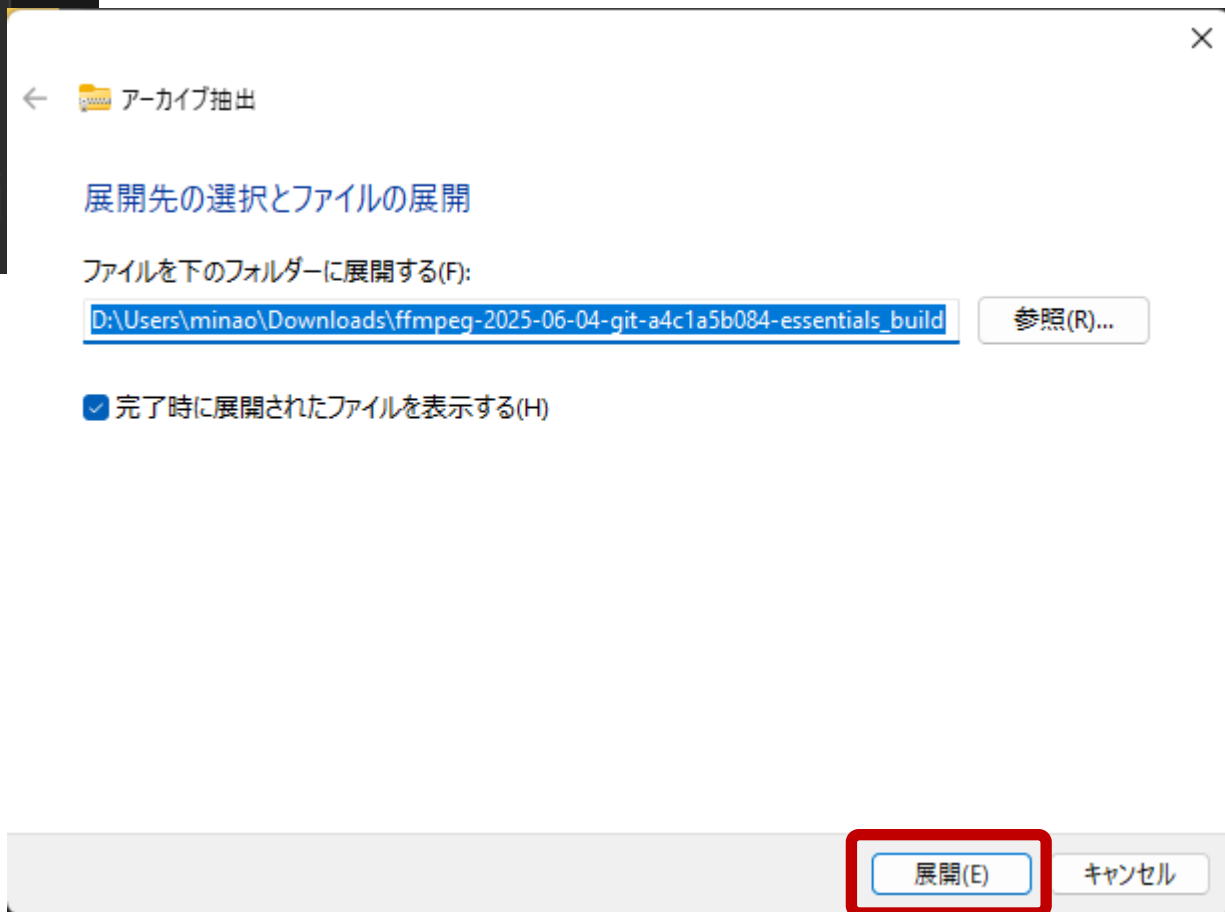
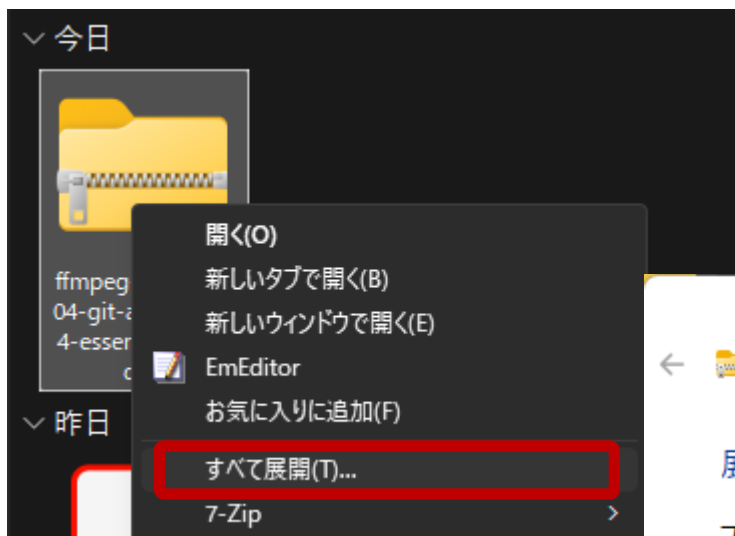
<https://www.gyan.dev/ffmpeg/builds/>

ffmpegの最新版をダウンロードします。

The screenshot shows the 'CODEX FFMPEG' website. The page has a header with 'gyan.dev' and 'CODEX FFMPEG'. Below the header, there are tabs for 'home' and 'builds'. The main content area contains text about the packages and instructions for downloading them. A red arrow points from the text 'ffmpegの最新版をダウンロードします。' to a red box highlighting the 'latest git master branch build' section. The table below shows the build details.

latest git master branch build	version: 2025-06-04-git-a4c1a5b084
ffmpeg-git-essentials.7z	.ver .sha256
ffmpeg-git-full.7z	.ver .sha256

FFmpegをダウンロードします



FFmpegをダウンロードします

<https://www.gyan.dev/ffmpeg/builds/>

C:\Users\minao\Downloads\ffmpeg-2025-06-04-git-a4c1a5b084-essentials_build\ffmpeg-2025-06-04-git-a4c1a5b084-essentials_build\bin

このフォルダにffmpeg.exeプログラムがあります。

▼ 先週

 ffplay.exe	2024/07/11 19:06	アプリケーション	84,440 KB
 ffprobe.exe	2024/07/11 19:06	アプリケーション	84,493 KB
 ffmpeg.exe	2024/07/11 19:06	アプリケーション	84,655 KB

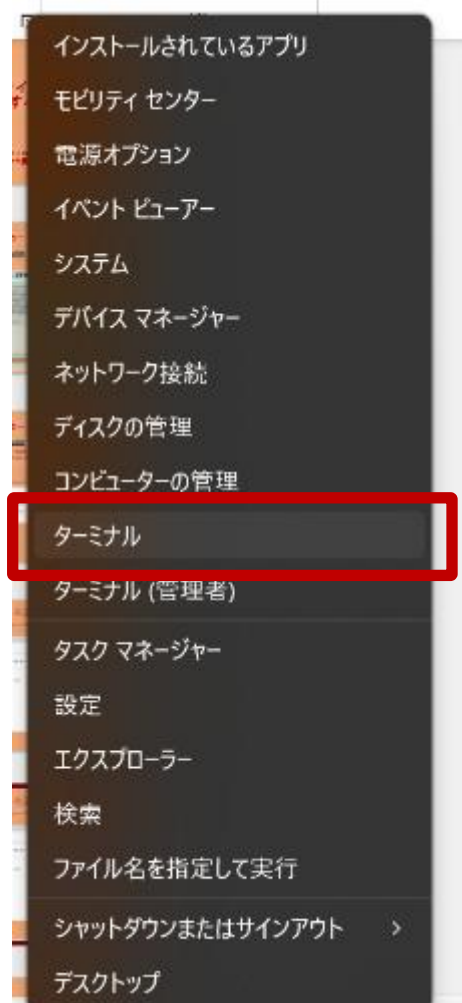
ffmpegによる動画の作成

frame0.jpg〜が入ったフォルダに、ffmpeg.exeをコピーします。



ffmpegによる動画の作成

ターミナルを立ち上げます。



ffmpegによる動画の作成

SDカードのフォルダにcdします。



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

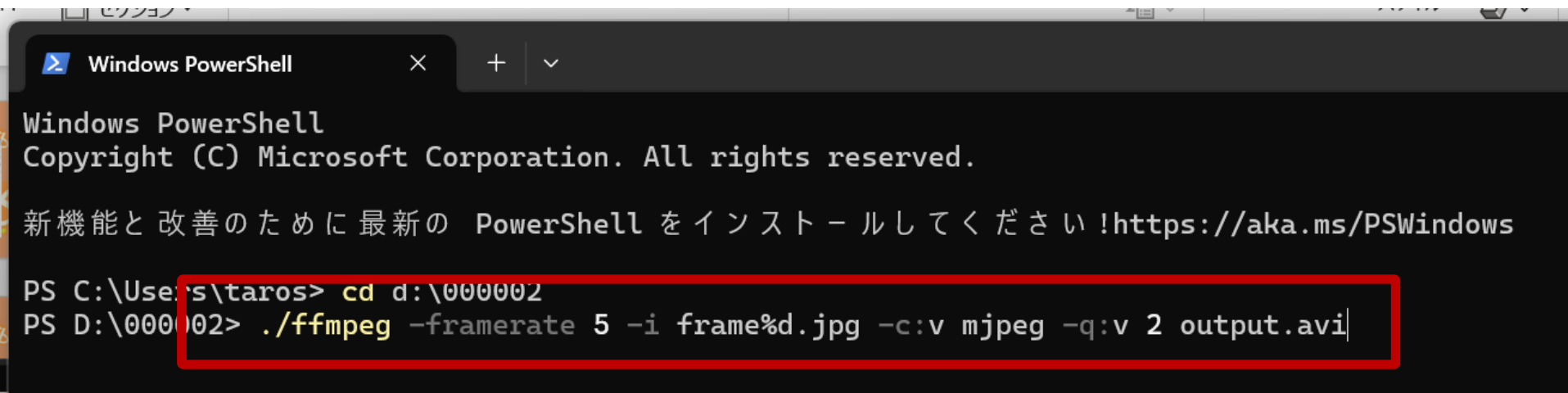
新機能と改善のために最新の PowerShell をインストールしてください!https://aka.ms/PSWindows

PS C:\Users\taro> cd d:\000002
PS D:\000002> |
```

SDカードが、Dドライブで、Jpegのフォルダが000002の場合

ffmpegによる動画の作成

ffmpegコマンドを実行します。



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

新機能と改善のために最新の PowerShell をインストールしてください!https://aka.ms/PSWindows

PS C:\Users\taros> cd d:\0000002
PS D:\0000002> ./ffmpeg -framerate 5 -i frame%d.jpg -c:v mjpeg -q:v 2 output.avi
```

> ./ffmpeg -framerate 5 -i frame%d.jpg -c:v mjpeg -q:v 2 output.avi

これで、output.avi というaviファイルが生成されます。

リンク先でコピーできます

エントレス 撮影プログラム

カメラプログラム

電源が落ちるまで、撮影し続けるプログラムを作りました。

EternalCapture.inoです。

```
01:33:14.518 -> WiFiを無効化しました
01:33:14.518 -> SDカードがマウントされました
01:33:14.518 -> カメラの初期化をします
01:33:15.184 -> □[0;31mE (1669) camera: camera probe ok□[0m
01:33:15.226 -> □[0;31mE (1670) camera: Setting frame size to 320x240□[0m
01:33:15.226 -> カメラが初期化されました
01:33:15.226 -> カメラセンサーを取得します
01:33:15.226 -> PIXFORMAT_JPEGです
01:33:15.226 -> ホワイトバランスのモード設定
01:33:15.226 -> 4枚を空撮影、ホワイトバランスを安定されるため
01:33:15.884 -> フォルダ番号最大値検索
01:33:15.993 -> キャプチャ開始
```

電源を入れると、フォルダを作成して、600枚の写真を撮ります。
600枚撮り終わると、新規にフォルダを作成して、600枚の写真を撮ります。
これを永遠に繰り返します。
撮影中は、青色LEDが高速点滅しています。
5FPSですので、2分間の動画が撮影されていきます。

本番に向けて、頑張ってください。

ご清聴ありがとうございました。